

RAPPORT DE PROJET

AiTunes

RÉALISÉ PAR

Byron CHOLET & Meryem IDBOUCAIR

ENCADRÉ PAR

M. Mehdi LHOMMEAU

Année de formation

Systèmes Automatisés et Génie Informatique
2024 / 2025

REMERCIEMENTS

Avant de débiter ce rapport, nous souhaitons exprimer notre sincère reconnaissance envers les personnes et structures nous ayant accompagné dans le cadre de ce projet. Nos remerciements se dirigent vers Polytech Angers et ses équipes pédagogiques et administratives, dont l'implication a été essentielle dans la mise en œuvre et le bon déroulement des projets.

Nous remercions particulièrement M. LHOMMEAU Mehdi, notre professeur référent, pour son accompagnement, son soutien et l'intérêt qu'il a constamment manifesté pour nos démarches.

Notre gratitude s'étend enfin à l'équipe du Laboratoire d'Étude et de Recherche Informatique d'Angers (LERIA), qui nous a témoigné sa confiance en nous accordant les moyens de calculs essentiels à la mise à bien de ce projet.

RÉSUMÉ / ABSTRACT

A. Version Française

Notre projet explore les capacités génératives des modèles Auto-Encodeurs Variationnels (VAE) dans un cadre de création musicale. A la différence de Jukebox par OpenAI, qui utilise des signaux audio bruts, notre approche repose sur une représentation des données basée sur des spectrogrammes logarithmiques projetés sur l'échelle des mels. En s'appuyant sur les bibliothèques PyTorch (apprentissage automatique) et Librosa (traitement audio numérique), nous avons développé un écosystème flexible facilitant l'automatisation des phases d'entraînement, de test et de mise en production sur diverses architectures (classiques, convolutives, résiduelles et discrètes). Cette approche automatique et graphique nous a permis de mettre au point des scénarios sur différents datasets (MNIST et CIFAR10 pour les images, GTZAN et FMA pour les musiques) avec des stratégies d'entraînement innovantes que nous détaillerons. Nous revenons dans ce rapport sur les atouts et les faiblesses révélés par nos expérimentations grâce à des techniques d'illustration visuelle des représentations latentes faites par nos modèles. Nous y comparons également les performances de reconstruction et les capacités génératives des architectures afin d'évaluer leur pertinence dans un contexte de génération musicale.

B. English Version

Our project explores the creative capabilities of Variational Auto-Encoder models (VAE) when applied to music-generation tasks. Contrary to OpenAI's Jukebox model, which uses raw audio signals, our approach relies on a logarithmic, mel-scale spectrogram representation of this data. Using libraries like PyTorch (for deep-learning) and Librosa (for numeric audio processing), we have developed a flexible ecosystem to facilitate the automation of training, testing and release phases on diverse architectures (standard, convolutional, residual and discrete). This automatic and graphical approach has enabled us to finetune scenarios based on different datasets (MNIST and CIFAR10 for images, GTZAN and FMA for music) with innovative training strategies which we'll detail. In this report we study the strengths and weaknesses revealed during our experiments using visual representation techniques of our models' latent mapping. We also provide a comparison of the architectures' reconstruction performance and generative abilities in order to evaluate their relevance in a music generation setting.

SOMMAIRE

I. INTRODUCTION.....	4
A. Mise en contexte.....	4
B. Objectifs et outils.....	4
C. Méthodologie appliquée.....	4
D. Présentation du plan.....	5
II. DÉVELOPPEMENT DU PROJET.....	5
A. Représentation numérique des signaux.....	5
1. Conversion Analogique / Numérique.....	5
2. Extraction de caractéristiques et reconstruction des signaux.....	6
3. Interface de manipulation audio.....	8
B. Comprendre les auto-encodeurs.....	9
1. Cas d'un auto-encodeur classique.....	9
a) Description de l'architecture.....	9
b) Implémentation d'un auto-encodeur avec PyTorch.....	10
c) Phases d'entraînement, d'évaluation et de génération.....	11
d) Programmation des processus d'entraînement et d'évaluation.....	12
2. Auto-Encodeur Variationnel (VAE).....	14
a) Description de l'architecture.....	14
b) Implémentation d'un VAE avec PyTorch.....	16
c) Application au dataset MNIST.....	16
3. Architecture convolutive (Convolutional VAE).....	18
a) Description de l'architecture.....	18
b) Implémentation d'un VAE convolutif.....	20
c) Application au dataset SINEWAVE.....	21
4. Architecture résiduelle (ResNet).....	22
a) Description de l'architecture.....	22
III. Auto-Encodeur Variationnel à Quantification Vectorielle (VQ-VAE).....	23
A. Description de l'architecture.....	23
B. Implémentation et application à un dataset de musiques réelles.....	24
D. Systèmes annexes et interfaces graphiques.....	25
IV. PERSPECTIVES D'ÉVOLUTION.....	27
A. Applications génératives.....	27
B. Enrichissement de la plateforme web.....	28
V. CONCLUSION ET MOT DE FIN.....	28
BIBLIOGRAPHIE.....	30
ANNEXE 1 : Fonction d'entraînement et d'évaluation.....	31
ANNEXE 2 : Implémentation de la classe LinearExperiment.....	32
ANNEXE 3 : Auto-Encodeur variationnel avec PyTorch.....	33
ANNEXE 4 : Implémentation du VAE convolutif.....	34
ANNEXE 5 : Application à un dataset de musiques réelles.....	35

I. INTRODUCTION

A. Mise en contexte

Tous les étudiants en deuxième année de cycle ingénieur à Polytech Angers ont l'opportunité de prendre part à un projet de groupe dans le but d'explorer des notions que le programme ne couvre pas nécessairement. Pour la filière Systèmes Automatisés et Génie Informatique (SAGI), les thèmes tournent autour du développement web, de la robotique, de la réalité virtuelle, de l'intelligence artificielle. Dans tous les cas, un effort d'organisation, de recherche, d'expérimentation et une vision critique sont attendus lors de nos travaux, des compétences transversales essentielles à notre futur rôle d'ingénieur.

Au cours de ces cent heures de projet réparties entre janvier et avril 2025, nous avons travaillé en binôme sur le sujet de la création musicale par intelligence artificielle. Ce domaine nous attire particulièrement en vue de l'essor actuel des solutions basées sur des algorithmes d'apprentissage profond génératif (MusicVAE par Google^[1], StyleGAN par NVIDIA^[2], ChatGPT par OpenAI^[3]). Nous avons alors saisi l'opportunité de redécouvrir leurs fondamentaux en créant notre propre solution d'intelligence artificielle générative.

B. Objectifs et outils

L'objectif du projet AiTunes était d'explorer les capacités génératives des modèles d'Auto-Encodeurs Variationnels (VAE) en les appliquant à un contexte musical avec une représentation spectrale des signaux audio. Cette approche diffère des modèles performants traditionnels qui utilisent plutôt des signaux audio bruts (Jukebox par OpenAI^[4]) ou des représentations symboliques MIDI (MusicVAE par Google).

Les concepts que nous avons exploré et qui sont détaillés dans ce rapport sont la représentation numérique et le traitement des signaux audio, ainsi que la programmation de réseaux de neurones profonds. Pour ce faire, nous avons notamment eu recours au langage de programmation Python, très populaire dans le domaine de la data science, et à diverses bibliothèques comme PyTorch (apprentissage profond), Librosa (traitement audio), Matplotlib (visualisation des données) et bien d'autres.

Pour une meilleure accessibilité et une expérience plus agréable, nous nous sommes également donné le défi de créer des interfaces utilisateur autour de notre solution principale. La première sert à gérer les modèles et orchestrer des programmes d'entraînement, et la seconde propose une interface web pour une mise en production dans le cas où les résultats le permettraient.

C. Méthodologie appliquée

Ayant tous les deux déjà pratiqué à très petite échelle le développement de réseaux de neurones avec Tensorflow, nous étions conscients des contraintes d'un tel projet, notamment au niveau des longues durées d'entraînement. Ainsi, lors de la première période, nous avons travaillé en collaboration sur la partie intelligence artificielle et réalisé nos tests sur des

datasets de petite taille pour valider nos architectures. Ensuite, lorsque les modèles se sont complexifiés et les temps de calcul ont inévitablement augmenté, nous nous sommes séparés les tâches pour travailler en parallèle sur les parties web et intelligence artificielle.

La synchronisation de nos contributions s'est effectuée par le biais des outils de contrôle de version Git et GitHub. Les dépôts contenant le code source du projet sont disponibles publiquement sur le projet AiTunes^[5] pour ce qui concerne le back-end (PyTorch, Flask) et AiTunes Web Project^[6] pour l'aspect front-end de la partie web (React).

D. Présentation du plan

Bien que nous ayons tenté de maintenir l'ordre temporel de notre travail à l'occasion de ce projet, nous avons pris quelques libertés pour organiser l'introduction de concepts de manière plus agréable et logique. Ainsi, nous commencerons par évoquer la représentation numérique des signaux et les systèmes de représentation et de traitement audio que nous avons mis en place. Nous aborderons ensuite la partie majeure du projet concernant les réseaux de neurones, les auto-encodeurs ainsi que toutes les variantes avec lesquelles nous avons expérimenté. Pour chaque modèle et architecture, nous détaillerons quelques exemples pertinents parmi les plus de 40 scénarios concrets que nous avons mis en place. Nous reviendrons ensuite sur d'autres essais annexes que nous avons réalisés mais qui ne trouvaient pas leur place dans les sections précédentes. Nous poursuivrons avec une explication plus détaillée des systèmes d'abstraction mis en place et des interfaces créées. Enfin, nous terminerons sur les directions que nous aurions aimé suivre avec ce projet si nous avions eu plus de temps et nous établirons un bilan de notre ressenti à l'issue de ces quatre mois de projet.

II. DÉVELOPPEMENT DU PROJET

A. Représentation numérique des signaux

1. Conversion Analogique / Numérique

Avant de débiter le développement de réseaux de neurones, il est intéressant de comprendre comment des signaux audios analogiques sont représentés numériquement pour être interprétés par un ordinateur. Ce procédé s'appelle l'échantillonnage et consiste à mesurer l'amplitude du signal continuellement perçu à intervalles de temps réguliers. Cette fréquence est nommée le **taux d'échantillonnage**, ou *sample rate* en anglais, et est de l'ordre de plusieurs dizaines de milliers de Hertz (échantillons par seconde).

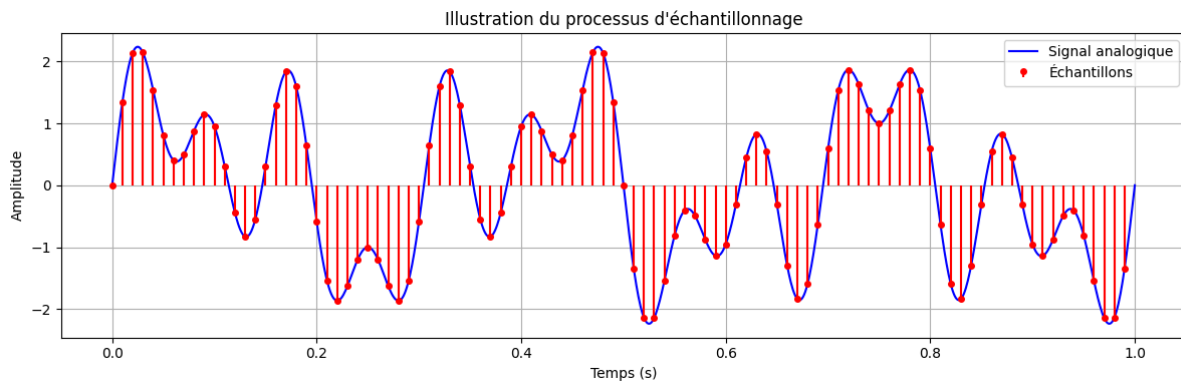


Figure 1: Illustration du processus d'échantillonnage depuis un signal analogique

Dans sa forme la plus simple (PCM, *Pulse-code modulation*), le format de fichier WAV contient les valeurs non-compressées des échantillons ainsi que quelques métadonnées additionnelles comme le taux d'échantillonnage et le nombre de canaux. Un codec peut ensuite être appliqué lors de la sauvegarde vers un fichier, comme le MP3 qui supprime les éléments généralement non-audibles par l'oreille humaine. Cependant, tout ceci est désormais automatiquement pris en charge par les bibliothèques de traitement audio classique.

Notre première création pour ce projet fut un assortiment de fonctions Python facilitant la génération de fichiers WAV simples. Une version minimaliste est présentée ci-dessous, où une unique fonction sinusoïdale d'une certaine fréquence est appliquée à un vecteur temporel régulier. Le signal résultant est ensuite converti en entiers 16 bits afin de respecter le format PCM standard, puis sauvegardé sous forme de fichier WAV.

```
import numpy as np
from scipy.io.wavfile import write

def simple_sine_wave(amp=0.5, duration=1, frequency=500, sample_rate=44100):
    t = np.linspace(0, duration, int(sample_rate * duration), endpoint=False)
    waveform = amp * np.sin(2 * np.pi * frequency * t)
    waveform_pcm = np.int16(waveform * 32767)
    write("out.wav", sample_rate, waveform_pcm)
```

Figure 2: Code de démonstration pour la génération d'ondes sinusoïdales vers un fichier WAV

Ces fonctions nous laissent facilement créer des extraits audio simples comme des combinaisons d'ondes sinusoïdales de fréquences contrôlées, une fonctionnalité qui nous sera utile ultérieurement dans ce rapport.

2. Extraction de caractéristiques et reconstruction des signaux

Nous l'avons déjà évoqué, certains modèles comme **Jukebox** sont entraînés sur des signaux audio bruts, c'est-à-dire sur les échantillons d'amplitude extraits directement de fichiers tels que les WAV. Ces séries temporelles unidimensionnelles peuvent être représentées sous forme

d'onde comme illustré sur la **figure 1**, mais lors de ce projet, l'idée était plutôt d'explorer l'extraction de caractéristiques au travers de spectrogrammes.

Les spectrogrammes sont une représentation en deux dimensions de l'amplitude du signal audio en fonction du temps et de la fréquence. Leur génération avec une bibliothèque de manipulation audio comme Librosa est triviale puisqu'elle implémente directement la transformée de Fourier à court terme (**STFT**, ou *Short-Time Fourier Transform*), qui permet d'analyser les composantes fréquentielles d'un signal dans des fenêtres temporelles. La qualité de cette représentation est facilement contrôlable en modifiant la taille (*window size*) et le pas (*hop length*) entre chaque fenêtre. Une petite fenêtre permet une meilleure résolution temporelle au détriment de la précision fréquentielle. Ce choix influence également la quantité de données à traiter, un équilibre à considérer selon l'objectif du réseau de neurones. Il est ensuite possible de réaliser l'approximation des signaux initiaux à partir d'un spectrogramme grâce à des algorithmes d'inversion comme **Griffin-Lim**, qui lui aussi est directement implémenté dans Librosa. Le défi principal de cette opération réside dans la reconstitution de la phase qui est perdue lors de la STFT, ce qui aboutit souvent à une sonorité robotique.

La perception humaine des sons étant plus sensible aux variations dans les fréquences basses, un changement commun est celui vers l'échelle de Mel qui compresse les hautes fréquences. La formule mathématique (par Douglas O'Shaughnessy, 1987^[7]) pour ce changement d'échelle est la suivante, où m est en mels et f en Hz:

$$m = 2595 * \log_{10}\left(1 + \frac{f}{700}\right)$$

De même, l'audition humaine est plus proche d'une perception logarithmique (en décibels) que linéaire en ce qui concerne l'intensité des signaux. Par exemple, un passage de 100 à 200 hertz sera perçu de manière similaire à une augmentation de 1000 à 2000 hertz.

Ainsi, puisque nous cherchons à produire des résultats plaisants pour l'oreille humaine, nous utiliserons pour la suite de ce projet des spectrogrammes mel sous échelle logarithmique (log-mel spectrogram) pour refléter toutes les particularités qui lui sont propres. Il devrait alors être plus simple pour nos modèles de trouver des patterns répétitifs dans les hautes fréquences, et de les compresser. Une comparaison des représentations abordées dans cette section est illustrée ci-dessous pour un signal combinant des ondes sinusoïdales de fréquences 512, 1024, 2048, 4096 et 8192 Hertz, avec une fréquence d'échantillonnage de 22050 Hz:

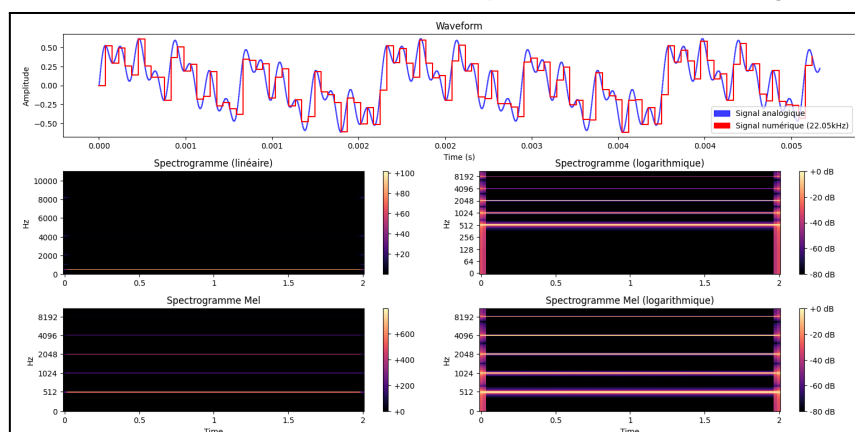


Figure 3: Comparaison des représentations spectrales de signaux audio

Une dernière représentation que nous aurions pu adopter pour ce projet est celle des *Mel-Frequency Cepstral Coefficients* (**MFCCs**), les coefficients d'une décomposition en cosinus discrets du spectrogramme mel. Tandis que la faible dimensionnalité peut être bénéfique pour une application d'apprentissage profond et que nos systèmes soient compatibles avec cette représentation, nous avons décidé de ne pas les utiliser en raison de certaines limites. En effet, si elles sont intéressantes pour des tâches de reconnaissance vocale, la perte d'information des MFCCs n'est pas négligeable lors d'une génération et/ou reproduction musicale de bonne qualité. A des fins d'illustration, nous avons tout de même représenté ces coefficients pour les comparer au spectrogramme log-mel d'une musique:

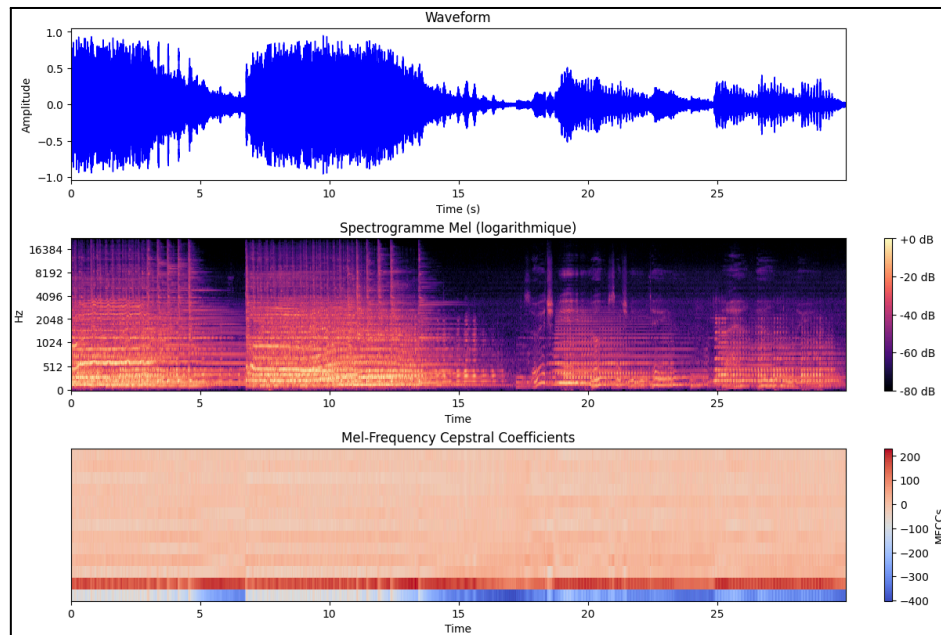


Figure 4: Visualisation des Mel-Frequency Cepstral Coefficients (MFCCs)

3. Interface de manipulation audio

Bien que Librosa facilite grandement les transformations que nous venons d'aborder, nous avons estimé utile de développer une interface centralisée pour accéder rapidement aux propriétés de données audio, réduisant le risque d'erreur et améliorant la lisibilité dans le reste du code. Pour répondre à ce besoin, nous avons créé la classe **AudioProcessingInterface** qui offre un accès direct à toutes les caractéristiques utiles d'un audio avec un support pour les appels chaînés de fonctions et une couche d'abstraction permettant de générer facilement des visuels comme ceux présentés précédemment.

```
from aitunes.audio_processing import AudioProcessingInterface

i = AudioProcessingInterface.create_for(path_to_audio, mode="file", label="Données originales")
i2 = i.extract_window(10.0, method="start", label="Extrait de 10s")
i3 = i.copy(label="Extrait atténué").preprocess(lambda y: y * 0.5)
i.full_summary([i2, i3], n_mels=128, mfcc_features=13)
```

Figure 5.1: Code de démonstration de la classe **AudioProcessingInterface**

Cet extrait de code génère un rapport complet d'une comparaison entre trois versions du même fichier audio:

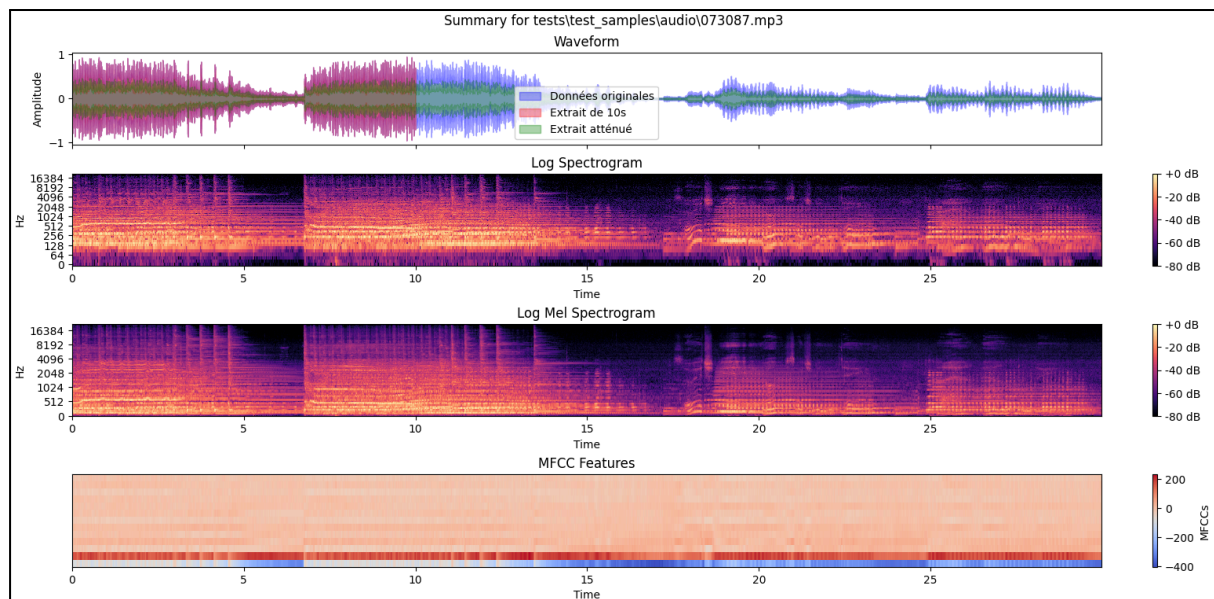


Figure 5.2: Comparatif de données audio réalisé grâce à la classe **AudioProcessingInterface**

Finalement, cette interface nous sert de point central pour convertir n'importe quel type de données (fichier brut, spectrogramme / mel-spectrogramme (logarithmique ou non), MFCCs) et y appliquer les opérations que l'on souhaite (prétraitement, sauvegarde, copie, extraction de fenêtre, ...). Son implémentation dès l'origine du projet nous a aidé à garder une base de code propre et compréhensible.

B. Comprendre les auto-encodeurs

1. Cas d'un auto-encodeur classique

a) Description de l'architecture

Maintenant que nous avons couvert la représentation numérique des données, il est nécessaire de comprendre comment un auto-encodeur fonctionne. Il s'agit d'une architecture de réseau de neurones dont l'objectif est d'apprendre une représentation optimisée des données en supprimant les informations redondantes. Dans le cas d'une mélodie, cette redondance peut se manifester par la répétition temporelle de tonalités comme des basses. Dans un spectrogramme, qui n'est finalement qu'une image en nuances de gris, cela s'illustre par une variation prévisible de l'intensité d'un pixel sur une certaine zone.

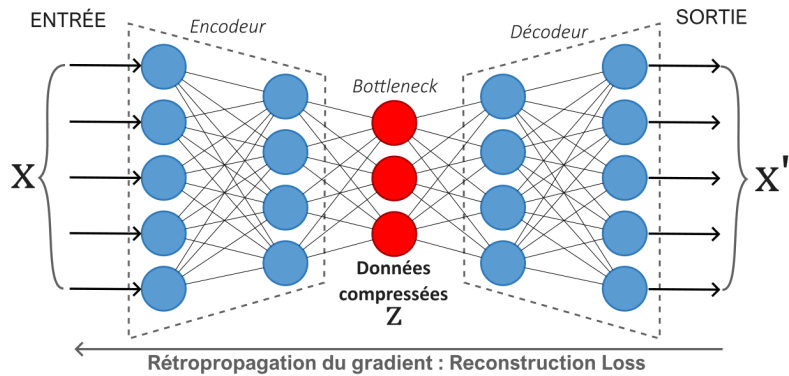


Figure 6: Schéma de l'infrastructure d'un auto-encodeur classique

Deux composants principaux constituent un auto-encodeur classique:

- L'encodeur réalise une compression représentative des données originales x vers une faible dimension appelée le *bottleneck*, ou goulot d'étranglement en français. Les couches entre les données d'entrée et le bottleneck sont les couches intermédiaires (*hidden layers* en anglais). Les codes issus de l'encodeur appartiennent au domaine z , l'espace latent du modèle.
- Le décodeur reçoit les codes latents produits par l'encodeur et les utilise pour reconstituer une approximation x' des données originales.

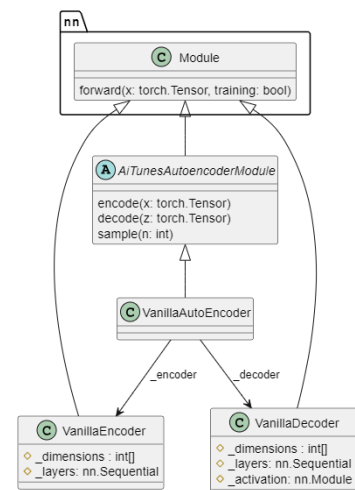
Cette architecture peut s'avérer très performante pour des tâches telles que la compression et la correction d'anomalies, comme pour une image ayant été dégradée par du bruit numérique. Cependant, les auto-encodeurs sont dénués de propriétés génératives à cause de leur espace latent déterministe (une même entrée produira un résultat identique) et de la structure même de cet espace latent qui n'est ni continue, ni interpolable. Ce sont alors des besoins que nous essaierons de résoudre dans les parties suivantes en abordant des alternatives aux auto-encodeurs classiques. Une illustration de ces propriétés est proposée plus loin dans ce rapport par la **figure 11** modélisant l'organisation de l'espace latent d'un auto-encodeur entraîné sur le dataset MNIST.

b) Implémentation d'un auto-encodeur avec PyTorch

Nous pouvons désormais nous intéresser à l'implémentation d'un auto-encodeur en Python grâce à PyTorch et Numpy. Pour ce faire, nous avons créé trois classes de type **nn.Module**, la classe de base pour tout module dans PyTorch. Les classes **VanillaEncoder** et **VanillaDecoder** construisent dynamiquement une séquence (**nn.Sequential**) de couches linéaires (**nn.Linear**) suivies de fonctions d'activation **nn.ReLU** pour rompre la linéarité des opérations. La définition de ReLU est la suivante:

$$f(x) = 0, \forall x \leq 0$$

$$f(x) = x, \forall x \geq 0$$



La classe **VanillaDecoder** ajoute également la possibilité de définir une fonction d'évaluation différente à la sortie, comme une activation **nn.Sigmoid** pour des résultats entre 0 et 1, ou **nn.Tanh** pour des résultats entre -1 et 1. Enfin, la classe **VanillaAutoEncoder** est constituée d'une instance de chacune des classes précédentes. Un diagramme UML de cette portion du code pourrait être celui ci-contre:

Puisque cette architecture restera essentiellement la même tout au long du projet, nous avons également créé une classe abstraite **AiTunesAutoEncoderModule** qui regroupe toutes les fonctions qu'un auto-encodeur devra implémenter. Dans le cas présent, nous avons:

- **encode(x)**: Passe le tenseur x à l'encodeur et retourne le code latent produit.
- **decode(z)**: Passe le tenseur z au décodeur et retourne la reconstitution réalisée.
- **forward(x, training)**: Reçoit les données originales en entrée, les fait passer par l'encodeur puis le décodeur, et retourne les données reconstituées ainsi que le code latent intermédiaire pour le réutiliser plus tard.

Avec ces classes, nous pouvons par exemple créer un auto-encodeur recevant des données en cinq dimensions. Ces données passent ensuite dans l'encodeur qui est composé de quatre couches linéaires produisant des tenseurs de dimensions 8, 7, 6 puis 3, chacune séparées d'une activation **ReLU**. Ce code latent en trois dimensions est ensuite donné au décodeur qui, à l'inverse, le décompresse en tenseurs de dimensions successives 6, 7, 8, puis 5:

```
from aitunes.modules import VanillaAutoEncoder

model = VanillaAutoEncoder(input_shape=5, hidden_layer_dimensions=[8, 7, 6], latent_dimension=3)
```

Figure 7: Exemple de code pour la création d'un auto-encodeur classique

c) Phases d'entraînement, d'évaluation et de génération

La phase d'entraînement pour un auto-encodeur classique consiste à présenter au modèle une série de valeurs x en entrée (exemple: des images) et de la comparer avec la série de résultats x' produits. Une *fonction d'évaluation* est alors appelée pour comparer la fidélité de la reproduction par rapport à l'original. Dans ce projet, nous utiliserons souvent l'erreur quadratique moyenne (*Mean-Squared Error*, ou **MSE**). La formule est la suivante pour une série de n valeurs, x étant les données originales et $\hat{x}$ les données reproduites par le décodeur (exemple : l'intensité des pixels d'une image).

$$MSE = \frac{1}{n} \times \sum_i^n (x_i - \hat{x}_i)^2$$

En réalité, cette fonction d'évaluation peut être n'importe quelle fonction (ou combinaison de fonctions) mathématique différentiable. Le gradient est ensuite calculé et utilisé pour modifier légèrement le poids de chacune des connexions entre les nœuds du modèle, de sorte à mieux s'adapter à la série de données dans le sens de la minimisation de l'erreur. Cet algorithme

d'apprentissage de base est celui du gradient stochastique, dont nous avons utilisé la variante **Adam**^[8] pour une meilleure stabilité d'entraînement.

Une époque (*epoch* en anglais) se termine lorsque le modèle a été entraîné sur l'ensemble du dataset prévu à cet effet. Les lots (*batches* en anglais) sont alors mélangés et une nouvelle époque commence. Cette méthode d'entraînement par lot permet de mieux représenter la variabilité entre plusieurs éléments, et ainsi d'obtenir un modèle plus généralisable.

La courbe d'évolution des erreurs calculées par la fonction d'évaluation peut être utilisée pour représenter les performances d'un modèle en fonction des époques. L'apparence d'une telle courbe est présente sur la figure suivante:

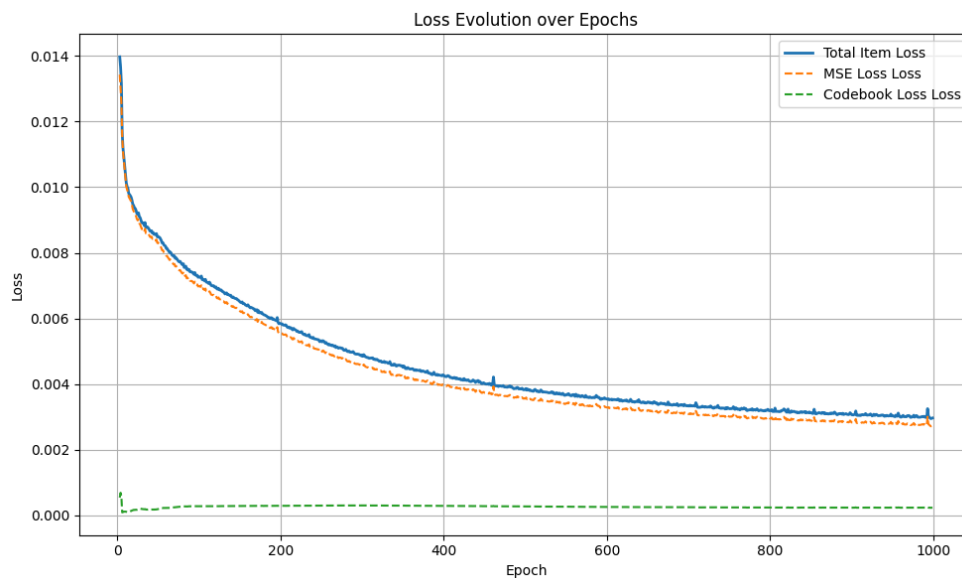


Figure 8: Modélisation de l'évolution des performances d'un auto-encodeur (ici, VQ-VAE)

Le suivi des performances sur le dataset d'entraînement permet de contrôler l'apprentissage du modèle. Toutefois, il reste crucial d'évaluer régulièrement son comportement sur un jeu de validation indépendant, car un bon score sur les données apprises peut dissimuler un **surapprentissage** (*overfitting*), limitant sa capacité à généraliser.

Une fois l'entraînement terminé, la génération de nouveaux éléments jamais rencontrés auparavant consiste à tirer un vecteur (aléatoire ou non) de même dimension que l'espace latent et de le passer directement au décodeur. Celui-ci se charge alors d'appliquer les transformations habituelles et de créer une nouvelle donnée en fonction des modèles mathématiques qui ont été appris. Comme nous l'avons vu, l'aspect déstructuré de l'espace latent d'un auto-encodeur classique le rend très difficile à prédire et à échantillonner.

d) Programmation des processus d'entraînement et d'évaluation

Comme pour la classe **AiTunesAutoEncoderModule**, les étapes d'entraînement restent sensiblement les mêmes entre les architectures et les datasets utilisés. En comprenant cela,

nous avons donc décidé de généraliser le processus au travers d'une classe abstraite extensible **AutoencoderExperiment**. Les arguments attendus dans le constructeur sont un nom (*string*), une architecture de modèle PyTorch (*AiTunesAutoEncoderModule*), un chemin vers le point de sauvegarde des poids du modèle (*string*), une fonction d'évaluation (*nn.Module*) et l'optimiseur utilisé pour l'entraînement (toujours Adam dans le cas de ce projet, avec des vitesses d'apprentissage différentes selon les cas). Les fonctions principales exposées par cette classe sont:

- **_load_weights** et **_save_weights**: Charger et sauvegarder les poids du modèle et de l'optimiseur depuis un fichier de type **.pth**, le format utilisé par PyTorch.
- **save_when**: Ajoute une fonction qui, lorsqu'elle est positive, déclenche la sauvegarde des poids du modèle.
- **train**: Lance le processus d'entraînement pour un certain nombre d'époques et crée des sauvegardes intermédiaires du modèle.
- **eval**: Réalise une évaluation du modèle sur des données jamais vues lors de l'entraînement et affiche le résultat dans la console.
- **next_batch (abstraite)**: Un générateur Python (afin de minimiser l'utilisation de la mémoire vive et la réserver à PyTorch) retournant des lots de données mélangés extraits du dataset d'entraînement ou d'évaluation.
- **interactive_evaluation (abstraite)**: Permettre une évaluation interactive du modèle en affichant en temps réel des comparaisons entre les données initiales et les reconstitutions. Cette visualisation dépend du type de données traitées (vecteurs, images, musiques), d'où son statut abstrait.

Un extrait commenté du code Python pour les fonctions **train** et **eval** est disponible à l'[Annexe 1](#). Pour une meilleure clarté du code, toutes les fonctionnalités concernant l'affichage dans la console et la sauvegarde périodique des modèles sont placées dans une instance de la classe **AutoencoderExperimentSupport**.

Dans l'objectif de valider le fonctionnement de notre implémentation pour un auto-encodeur classique, nous avons créé un dataset artificiel 2000 vecteurs en trois dimensions qui ont été augmentés en cinq dimensions par diverses opérations mathématiques non linéaires. Une implémentation partielle de la classe **LinearExperiment** correspondant à ce dataset est proposée en [Annexe 2](#). Nous pouvons désormais compléter le programme de la figure 7 avec une syntaxe facilement lisible pour mettre en application le modèle créé:

```

import torch.optim as optim
from aitunes.modules import VanillaAutoEncoder
from aitunes.experiments.cases import LinearExperiment
from aitunes.utils.loss_functions import create_mse_loss

model = VanillaAutoEncoder(input_shape=5, hidden_layer_dimensions=[8, 7, 6], latent_dimension=5)
loss = create_mse_loss(reduction='mean')
optimizer = optim.Adam(model.parameters(), lr=0.001)

experiment = LinearExperiment(model, weights_path='model.pth', loss=loss, optimizer=optimizer)
experiment.save_every(epoch_count=100, root_folder='.')

experiment.train(epochs=300)
experiment.evaluate()
experiment.interactive_evaluation()

```

Figure 9: Exemple de code pour la création d'un scénario d'entraînement

On obtient alors une courbe d'entraînement similaire à la suivante (sauvegardée sous forme de fichier .csv) et une interface interactive:

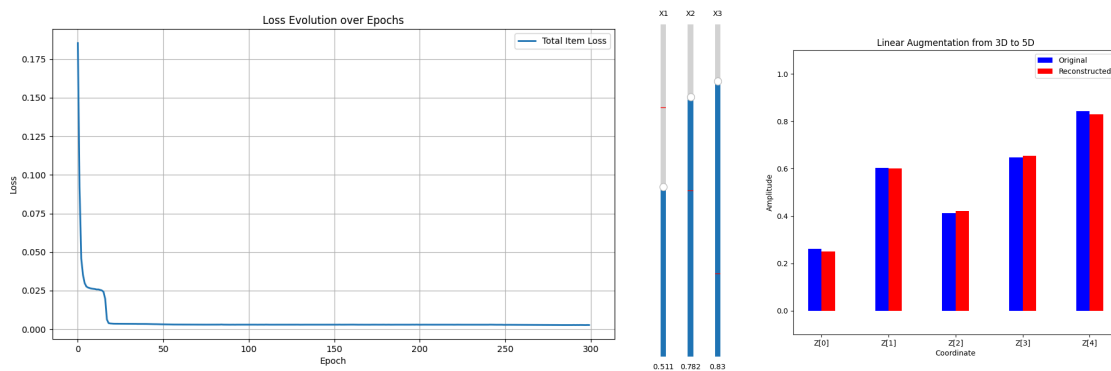


Figure 10: Résultat de l'entraînement pour un scénario simple

2. Auto-Encodeur Variationnel (VAE)

a) Description de l'architecture

L'auto-encodeur variationnel (VAE) est une évolution conceptuelle des auto-encodeurs classiques. Il se distingue par sa capacité à apprendre une distribution latente probabiliste des données d'entrée, l'élément manquant pour obtenir des propriétés génératives. On y retrouve l'infrastructure standard avec un encodeur et décodeur, mais leurs objectifs sont différents:

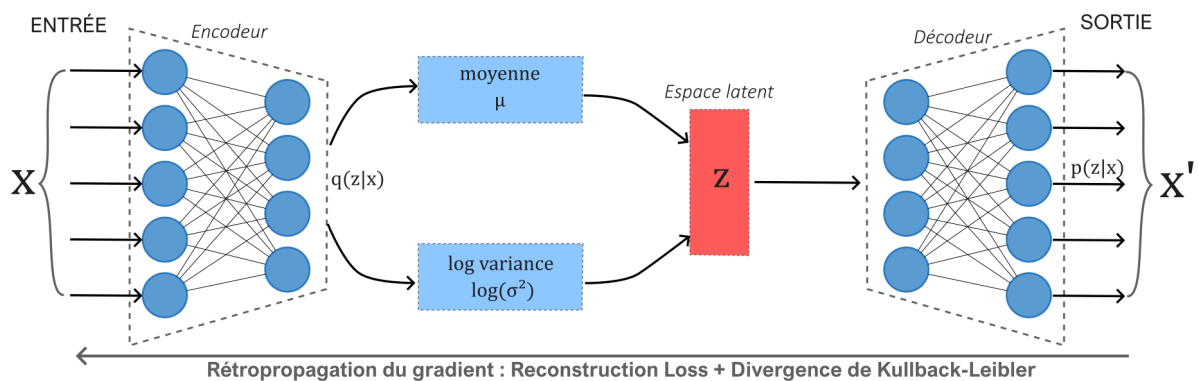


Figure 11: Infrastructure d'un Auto-Encodeur Variationnel (VAE)

Ici, l'encodeur doit toujours extraire les caractéristiques essentielles des données d'entrée x . En revanche, il ne produit pas un simple vecteur latent mais un vecteur de moyennes μ et un de log-variance $\log(\sigma^2)$, décrivant une distribution gaussienne multivariée depuis laquelle échantillonner un vecteur latent z .

Un point important à considérer est qu'il n'est pas judicieux de tirer une valeur z aléatoire depuis cette distribution $N(\mu, \sigma^2)$ puisqu'il s'agit d'un procédé non différentiable et par conséquent incompatible avec l'algorithme de descente de gradient. Pour pallier cette limitation, on fait appel à la technique appelée **reparameterization trick** (ou **reparameterization global estimator**) qui rend cet échantillonnage différentiable et détache l'aléatoire de la partie entraînable. Ce procédé repose sur les propriétés de la loi normale pour définir z :

$$z = \mu + \sigma \cdot \epsilon$$

avec $\epsilon \sim N(0, 1)$ une loi normale standard et $\sigma = e^{\frac{1}{2} \log(\sigma^2)}$.

Le rôle du décodeur est identique à celui d'un auto-encodeur classique qui est la prédiction d'une approximation de la donnée originale depuis le vecteur z reçu en entrée.

En ce qui concerne la fonction d'objectif du modèle, une évaluation de la qualité de la reproduction telle qu'une *MSE Loss* est combinée à la Divergence de Kullback-Leibler (*KL divergence*). Ce composant estime la proximité entre deux distributions de probabilité: la distribution postérieure approximée $q(z|x) \sim N(\mu, \sigma^2)$ générée par l'encodeur, et la distribution prior idéale $p(z) \sim N(0, 1)$ dans notre cas. L'objectif est de contraindre l'espace latent à suivre une distribution simple, standard et dense pour pouvoir échantillonner sans risque de rencontrer une zone « vide ». La formule analytique de la *KL divergence* pour une distribution postérieure de dimension n est la suivante:

$$KL(q(z|x) || p(z)) = -\frac{1}{2} \sum_{i=1}^n (1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2)$$

C'est notamment pour une meilleure stabilité numérique lors de ce calcul que l'encodeur produit un vecteur **log_var**. En effet, cela permet d'attendre un nombre réel quelconque de la part du modèle, tandis que le contraindre à générer une variance positive ajouterait une complexité supplémentaire par le biais d'une fonction d'activation comme un **softplus**.

Avec un modèle entraîné et un espace latent idéalement structuré en une distribution normale standard multivariée, la génération de nouveaux éléments peut se faire de différentes manières. La première, déterministe, consiste à tirer aléatoirement un vecteur latent selon une loi normale standard pour le passer directement au décodeur. La seconde, qui ne donnera pas le même résultat pour un même tirage, se fait par l'échantillonnage d'un vecteur de moyennes μ et d'un vecteur de log variances σ qui sont ensuite reparamétrisés et donnés au décodeur.

Enfin, il est aussi possible de prendre une combinaison linéaire de plusieurs codes latents connus et de les fournir au décodeur.

b) Implémentation d'un VAE avec PyTorch

Nous avons déjà établi une base solide avec nos classes prévues pour un auto-encodeur standard. Il nous suffit de créer une nouvelle classe **VariationalAutoEncoder** contenant un **VanillaEncoder**, un **VanillaDecoder** et deux couches denses **mu** et **log_var**. Nous pouvons maintenant implémenter les fonctions abstraites requises par **AiTunesAutoencoderModule**:

- **encode(x)**: Le tenseur **x** original est passé à l'encodeur qui génère un code. Celui-ci est passé à la couche **mu** et à la couche **log_var**, les deux tenseurs provenant de ces couches sont retournés.
- **decode(z)**: Passe le tenseur **z** au décodeur et retourne la reconstitution réalisée.
- **forward(x, training)**: Reçoit les données originales en entrée. Elles sont passées à la fonction **encode(x)** qui retourne **mu** et **log_var** pour la distribution postérieure multivariée de l'échantillon. Le reparameterization trick permet d'obtenir **z** qui est ensuite fourni au décodeur pour générer une estimation de la donnée originale. Cette fonction retourne le code latent, la reconstitution, mais également **mu** et **log_var** pour le calcul de la KL divergence.

La classe complète pour un Auto-Encodeur Variationnel et l'implémentation de la KL divergence est retranscrite en [Annexe 3](#).

c) Application au dataset MNIST

Le dataset MNIST est un ensemble de données de référence très utilisé dans le domaine de l'apprentissage automatique. Il contient 70 000 images de taille 28×28 pixels en niveaux de gris représentant des chiffres manuscrits de 0 à 9, dont 60 000 pour l'entraînement et 10 000 pour les tests. Dans cette application, les images sont compressées vers un espace latent de deux dimensions ce qui permet une visualisation sur un plan.

En vue de sa similarité avec la classe **LinearExperiment** présentée précédemment, il n'est pas nécessaire de rentrer dans les détails de la classe **MnistExperiment**. Les seules différences notables résident dans l'exploitation du package **torchvision** pour télécharger les datasets d'entraînement et d'évaluation, et l'utilisation de la classe de **DataLoader** intégrée à PyTorch pour générer les lots de données. La partie intéressante vient de la comparaison des espaces latents pour différents poids attribués à la KL Divergence (chaque graphique est généré individuellement avec un appel à **interactive_evaluation** de la classe **MnistExperiment** pour différents scénarios d'entraînement):

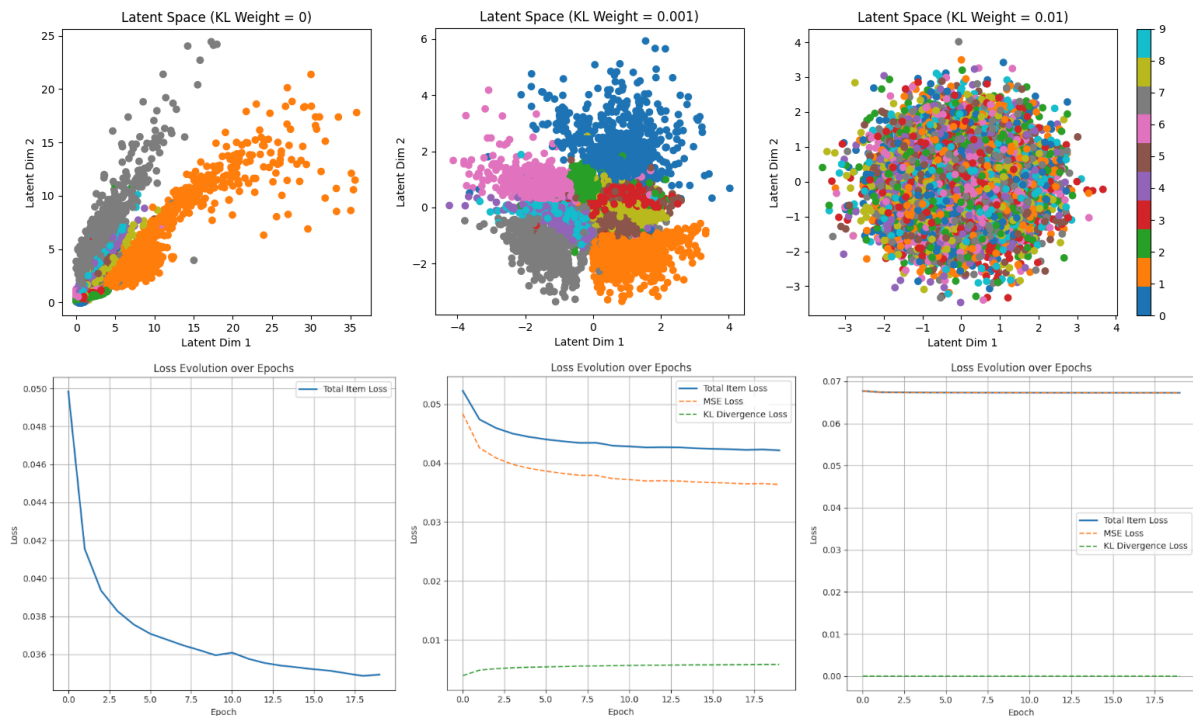


Figure 11: Comparaison de l'espace latent d'un VAE pour différents poids de KL Divergence

Cette modélisation est produite en enregistrant le vecteur latent de chaque élément du dataset d'entraînement et en le plaçant dans un plan avec une couleur représentative du chiffre dessiné sur l'image.

- Pour un poids de 0, soit l'équivalent d'un auto-encodeur classique, on remarque que l'espace latent du VAE n'est pas du tout organisé, avec des trous notables, des codes se chevauchant pour certains chiffres tandis que d'autres (ici 1 et 7) occupent une surface bien plus élevée. La fonction d'évaluation finale (*MSE Loss*) affiche une valeur de **0.0349**.
- Pour un poids de 0.001, l'espace latent est visiblement bien mieux organisé, avec quelques chiffres similaires se chevauchant (4 et 9, 3 et 8) mais une occupation plus compacte de l'espace latent. Les codes pour tous les chiffres se situent à une distance raisonnable du point (0, 0) et ne laissent paraître presque aucune zone vide dans un rayon de 3 autour du centre. Cette meilleure structuration n'est pas sans conséquence: le composant MSE caractérisant la qualité de reconstruction termine avec une valeur de **0.0364**, légèrement moins optimale que le premier cas même pour un dataset d'images simples.
- Pour un poids de 0.01, soit dix fois supérieur à l'essai précédent, l'espace latent postérieur suit une distribution normale presque parfaite, mais on ne distingue plus aucune structuration entre les différents chiffres. Cela se ressent également sur le graphique d'évolution de la fonction d'optimisation, dont le composant *KL Divergence* reste constamment autour de 0 tandis que le composant *MSE* n'évolue presque pas avec le temps. On peut donc en déduire que le poids de la *KL Divergence* est trop élevé dans ce cas, on ne peut rien tirer d'un modèle entraîné dans ces conditions.

Avec l'augmentation de la durée d'entraînement avec des datasets plus grands et des données plus complexes, il nous faut donc rester vigilant et choisir correctement le poids de la KL

Divergence pour privilégier une reconstruction correcte et ne pas perdre plusieurs heures de calcul. Par ailleurs, on parle d'un β -VAE lorsque la KL divergence est atténuée d'un certain coefficient β par rapport à la perte de reconstruction.

Enfin, pour mettre en évidence les capacités créatives d'un VAE, nous avons créé une interface avec Matplotlib pour générer en temps réel l'image associée à n'importe quel point de l'espace latent. Nous en avons enregistré une courte démonstration en vidéo^[9]. Grâce à cela, nous pouvons observer l'interpolation réalisée naturellement pour transitionner entre deux chiffres différents de manière cohérente. Dans cette vidéo, la génération est déterministe puisque le vecteur latent $[x_1, x_2]$ est passé au décodeur sans reparamétrisation.

3. Architecture convolutive (Convolutional VAE)

a) Description de l'architecture

Dans une architecture classique, tous les nœuds sont liés entre eux sans tenir compte de leur "distance". Ces liens entraînent des calculs supplémentaires et terminent souvent avec des poids proches de zéro en vue de leur importance presque nulle. Pour cette raison, nous nous sommes rapidement intéressés à une architecture utilisant des **couches de convolution** à la place des simples couches denses illustrées dans la figure 6. Cette transformation est particulièrement adaptée au cas des images et de la musique puisqu'elle met en relation des groupes de données proches et se concentre sur l'extraction de motifs locaux.

Comme illustré par la figure suivante, une convolution consiste à appliquer un filtre de taille $n \times m$ (ici $n = m = 3$) à la donnée d'entrée par le biais d'un produit scalaire. Une fois appliqué, le filtre est déplacé d'un certain pas (*stride* en anglais, ici égal à 1) et de répéter l'opération.

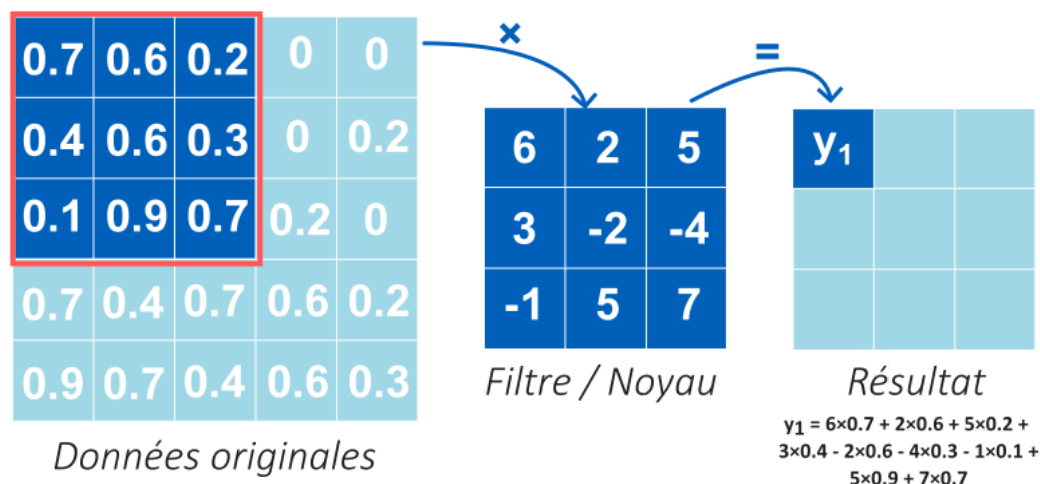


Figure 12: Schéma d'une couche convolutive de noyau 3x3 et de pas 1

Chaque filtre est appliqué à la totalité de l'image et le résultat est superposé sous forme de canal (*channel* en anglais) aux résultats des autres filtres.

Prenons une image carrée en nuances de gris de dimension $28 \times 28 \times 1$. Celle-ci est initialement composée d'un canal (l'intensité des pixels) de taille 28×28 . L'illustration ci-dessous représente le fonctionnement complet d'un auto-encodeur convolutif, dont la tendance sera d'augmenter progressivement le nombre de filtres tout en réduisant la dimensionnalité de chacun des canaux.

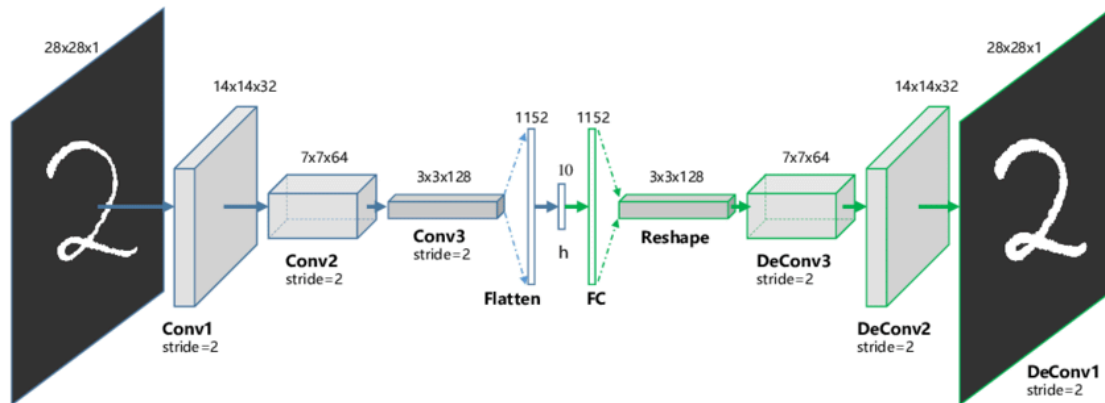


Figure 13: Schéma d'un auto-encodeur convolutif^[10]

Encodeur (zone de gauche): On débute avec une image x de dimension $28 \times 28 \times 1$. L'objectif de l'encodeur est toujours le même, compresser cette image en un vecteur plus petit tout en conservant les informations essentielles. Les étapes de compression sont les suivantes:

- **Conv1 (stride=2):** Application de 32 filtres de convolution avec un pas de 2, divisant la dimensionnalité de chaque canal par deux. On obtient donc 32 caractéristiques de l'image de dimension 14×14 .
- **Conv2 et Conv3 (stride=2):** Même opération, résultant en 128 caractéristiques de dimensions 3×3 .
- **Flatten:** Les composantes de ces caractéristiques sont aplaties en un unique vecteur de dimension $3 * 3 * 128 = 1152$ compatible avec une couche dense. Celle-ci réalise une combinaison linéaire de ces 1152 coefficients pour obtenir le vecteur latent z de dimension 10.

Décodeur (zone de droite): Le rôle du décodeur est de reconstruire l'image d'origine à partir de ce petit vecteur latent. Il est souvent symétrique à l'encodeur et doit produire un résultat de même dimensionnalité que l'image d'entrée pour que la fonction d'évaluation ait un sens (Il n'est pas logique de comparer un à un les pixels de deux images de dimension différente).

- **Unflatten:** Une couche dense prend le code latent et en construit un vecteur de dimension 1152, qui est déplié en un volume de taille $3 \times 3 \times 128$.
- **DeConv 3, DeConv 2, DeConv 1 (stride=2):** Les couches de déconvolution (ConvTranspose2d sur PyTorch) réalisent l'opération inverse à une convolution, en appliquant un nombre décroissant de filtres qui augmentent la dimensionnalité de chaque canal.

Le procédé précédent n'est généralement pas appliqué à un élément unique, mais plutôt à un lot d'éléments du dataset. Le lot de sortie peut être comparé pixel à pixel avec le lot d'entrée, et

une rétropropagation du gradient de cette évaluation est utilisée pour changer les poids des liaisons entre les nœuds et les coefficients des filtres de convolution.

b) Implémentation d'un VAE convolutif

Avec les bases vues précédemment pour un VAE, nous avons créé trois nouvelles classes pour s'adapter à l'architecture convolutive. La classe **CVAE** est une autre implémentation de **AiTunesAutoEncoderModule** et fait le lien entre un **ConvolutionalEncoder** et un **ConvolutionalDecoder**, deux objets construits depuis une liste de paramètres nécessaires à la création de couches convolutives (nombre et taille des filtres ainsi que le pas).

Une difficulté que nous avons rencontré concerne le décodeur. Pour une couche convolutive (**nn.Conv2d**), la formule décrivant la taille de la sortie s_{out} pour une entrée de taille s_{in} est la suivante:

$$s_{out} = \text{floor}\left(\frac{s_{in} + 2 \times \text{padding} - \text{dilation} \times (\text{kernel_size} - 1) - 1}{\text{stride}} + 1\right)$$

Pour une couche convolutive inverse (**nn.ConvTranspose2d**), le taille de la sortie est:

$$S_{out} = (S_{in} - 1) \times \text{stride} - 2 \times \text{padding} + \text{dilation} \times (\text{kernel_size} - 1) + p_i + 1$$

avec, dans notre cas, $\text{dilation} = 1$ et p_i un padding asymétrique. Cependant, la fonction *floor* indique que des tailles différentes d'entrée s_{in} peuvent résulter en une taille de sortie s_{out} identique, il est alors en fait très peu probable que l'empilation d'opérations inverses **ConvTranspose2d** ne produise un résultat de même taille que l'entrée. Pour remédier à ce problème, la plupart des projets présents en ligne calculent manuellement la correction asymétrique à rajouter à chaque couche.

En revanche, nous souhaitons conserver l'aspect dynamique de la génération de nos modèles qui accélère grandement nos tests. Aussi, nous avons créé une fonction qui calcule automatiquement le *padding asymétrique* à ajouter à chaque étape du modèle pour répartir de manière optimale l'ajout de données arbitraires. Cette fonction, qui supporte également une asymétrie différente en hauteur et en largeur (utile pour des données non carrées, comme des spectrogrammes), est la suivante:

```
def compute_transpose_padding(kernel_size: int, padding: int, stride: Union[int, tuple[int, int]], input_shape:
tuple[int, int], output_shape: tuple[int, int]):

    if not isinstance(stride, tuple):
        stride = stride, stride

    h_out = (input_shape[0] - 1) * stride[0] - 2 * padding + 1 * (kernel_size - 1) + 1
    w_out = (input_shape[1] - 1) * stride[1] - 2 * padding + 1 * (kernel_size - 1) + 1
    p_h, p_w = output_shape[0] - h_out, output_shape[1] - w_out

    return p_h, p_w
```

Figure 14: Code pour le calcul du padding asymétrique d'une couche **nn.ConvTranspose2d**

L'implémentation complète des classes **ConvolutionalDecoder** et **CVAE** est disponible en [Annexe 4](#). **ConvolutionalEncoder** n'a pas été incluse de par sa similarité avec **ConvolutionalDecoder**.

c) Application au dataset SINEWAVE

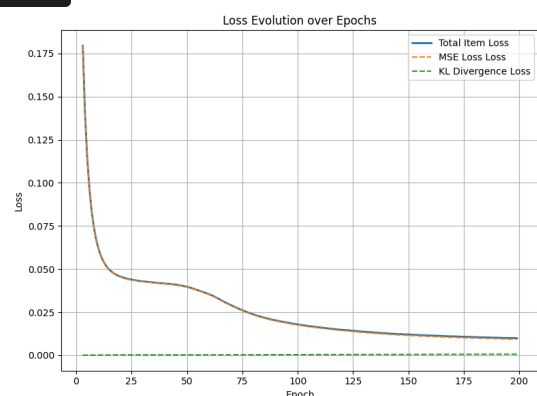
Parmi les nombreux scénarios que nous avons créés avec l'architecture convolutive simple, certains s'appliquent à un dataset de fichiers audio créé grâce aux fonctions simples de génération abordées plus tôt dans la partie 2.1. Composé de 2500 morceaux répartis sur cinq classes différentes, celui-ci nous a permis d'évaluer les performances de nos architectures et de nos systèmes (notamment l'intégration avec **AudioProcessingInterface**) plus rapidement que si nous l'avions appliqué à un dataset de musiques réelles. Puisque les procédés sont identiques pour tous les scénarios traitant de l'apprentissage sur des fichiers audio transformés en spectrogrammes, nous avons créé une classe commune **SpectrogramBasedAutoEncoderExperiment** étendant la classe **AutoencoderExperiment** abordée précédemment. Voici comment se déroule la mise en place d'un nouveau dataset:

1. Si le dataset est trouvable à un certain chemin d'accès, le scénario est prêt et l'on peut passer à l'entraînement.
2. Sinon, le dataset de fichiers audio est généré, soit par téléchargement depuis une base de données comme Kaggle^[11], soit par génération locale en Python.
3. Les fichiers audio individuels sont chargés grâce à notre **AudioProcessingInterface** et le spectrogramme log-mel est calculé, normalisé entre 0 et 1, puis retourné sous forme de tableau NumPy.
4. Périodiquement, ces spectrogrammes sont accumulés et sauvegardés dans un fichier (une petite partie est mise de côté pour l'évaluation) de base de donnée locale au format **.h5** grâce à la bibliothèque **h5Py**. Cela permet de limiter l'utilisation de la RAM et d'intercepter une quelconque erreur au plus tôt.

Lors de l'entraînement, les spectrogrammes log-mel normalisés originaux sont comparés à ceux issus du décodeur comme précédemment avec le dataset MNIST (MSE Loss + KL Divergence).

```
model = CVAE(
    input_shape=(1, *spectrogram_size),
    conv_channels=( 32, 64, 128, 256, 256),
    conv_kernels=( 3, 3, 3, 3, 1),
    conv_strides=( 2, 2, 2, 2, 1),
    latent_dimension=32
)
```

Pour un auto-encodeur variationnel convolutif de dimension latente 32, nous obtenons l'évolution ci-contre. De plus, nous avons créé une interface d'évaluation interactive des reconstructions visible sur la figure de droite, qui permet de comparer le spectrogramme original avec sa reconstruction, ainsi que de visualiser la représentation latente z avant d'être passée au décodeur:



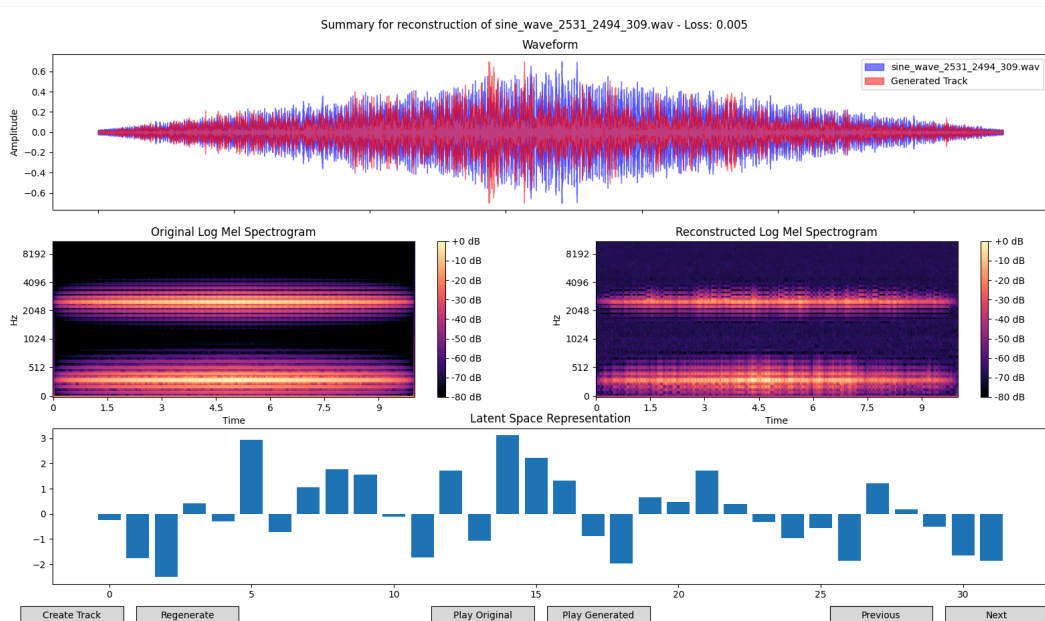


Figure 15: Comparaison des reconstructions réalisées par un auto-encodeur convolutif

4. Architecture résiduelle (ResNet)

a) Description de l'architecture

Il est naturel de penser qu'un réseau de neurones plus profond ait de meilleures capacités d'apprentissage puisqu'une augmentation du nombre de couches, et donc d'opérations, permet théoriquement de mieux approximer le comportement réel d'un phénomène. En réalité, il arrive souvent qu'un réseau profond s'adapte moins bien qu'un réseau simple : c'est le problème de dégradation du gradient (*Vanishing Gradient* en anglais)^[12]. Lors de la rétropropagation, les couches situées au début du réseau (celles proches des données d'entrée) reçoivent des gradients de plus en plus faibles à mesure que l'on remonte les couches depuis la sortie. Cela entraîne un apprentissage très lent, voire nul, dans ces premières couches, ce qui limite la capacité du réseau à capturer des représentations utiles des données.

Pour cette raison, nous nous sommes ensuite intéressés à l'architecture résiduelle, une innovation qui a transformé le domaine de l'intelligence artificielle pour les applications dont la complexité mathématique exige l'utilisation de réseaux profonds. En créant ce qui pourrait être perçu comme un « chemin alternatif », le gradient peut s'écouler sans perturbation à travers l'entièreté du réseau. La figure suivante présente la connexion réalisée au sein d'un bloc résiduel.

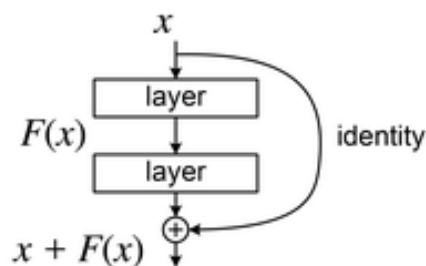


Figure 16: Schéma d'un bloc résiduel^[13]

Pour chacun de ces blocs pouvant être empilés, le modèle apprend maintenant une modélisation de la variation $F(x)$ entre l'entrée x et la sortie $x + F(x)$. Pour chaque couche plutôt que la transformation complète, une approche a le bénéfice supplémentaire de simplifier l'optimisation du réseau et donc d'accélérer l'apprentissage.

III. Auto-Encodeur Variationnel à Quantification Vectorielle (VQ-VAE)

A. Description de l'architecture

Le VQ-VAE (Vector Quantized Variational Autoencoder) propose une alternative à tous les modèles vus jusqu'à présent en introduisant une quantification vectorielle de l'espace latent. Concrètement, au lieu de projeter les données sur un espace continu, l'encodeur sélectionne une combinaison de vecteurs dans un dictionnaire discret (le *Codebook*). Cette approche permet une compression plus efficace et une encore meilleure structuration de l'espace latent. En particulier, cette discrétisation rend le modèle plus adapté à des données séquentielles ou symboliques comme la musique ou le langage, où une représentation par catégories est souvent plus pertinente que des variations continues.

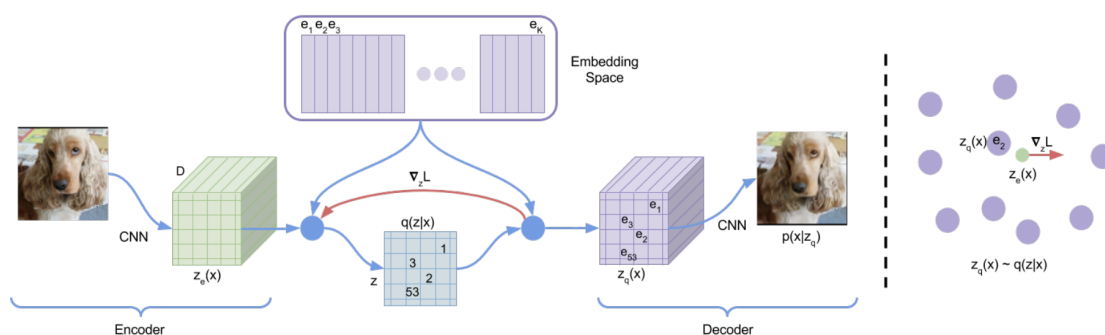


Figure 1: Left: A figure describing the VQ-VAE. Right: Visualisation of the embedding space. The output of the encoder $z(x)$ is mapped to the nearest point e_2 . The gradient $\nabla_z L$ (in red) will push the encoder to change its output, which could alter the configuration in the next forward pass.

Figure 17: Schéma de fonctionnement d'un VQ-VAE^[14]

Nous avons découvert ce type d'architecture en nous renseignant sur le modèle génératif Jukebox^[4] par OpenAI abordé plus tôt. Nous avons donc également implémenté certaines optimisations comme la technique de l'*Exponential Moving Average* introduite par Sonnet de Google Deepmind^[15], permettant de mettre à jour les vecteurs du dictionnaire de manière plus stable, en lissant leurs ajustements au fil du temps au lieu de s'appuyer uniquement sur les gradients directs. Une autre optimisation concerne le *Random Codebook Restart* qui consiste à remplacer les codes latents du dictionnaire discret lorsqu'ils ne sont pas ou très peu utilisés.

Comme expliqué sur cet article^[16], l'avantage des VQ VAE réside dans leur méthode de compression. En effet, tandis que les VAE traditionnels réduisent la dimensionnalité de la donnée en un unique vecteur de petite dimension, les VQ-VAE la transforment en un vecteur d'entiers où chaque valeur correspond à un vecteur discret. Cette méthode de fonctionnement

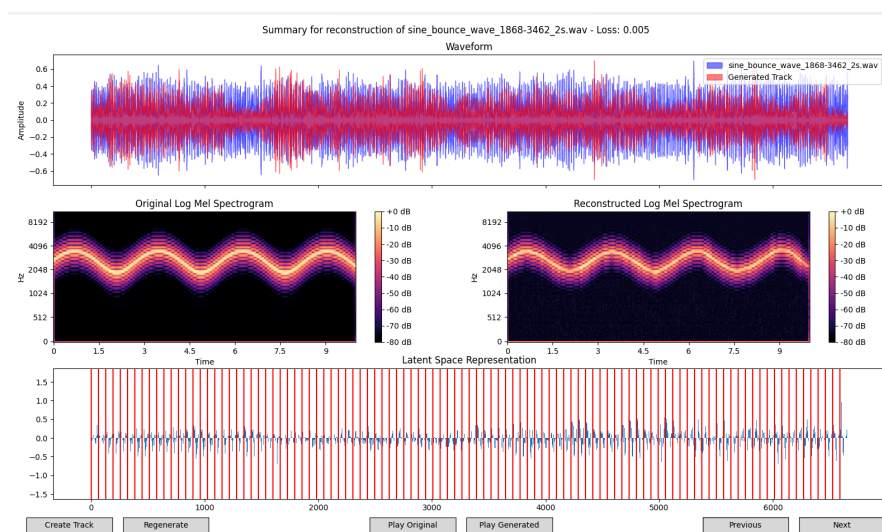
permet d'obtenir un niveau de compression très élevé en encourageant l'apprentissage de représentations symboliques.

B. Implémentation et application à un dataset de musiques réelles

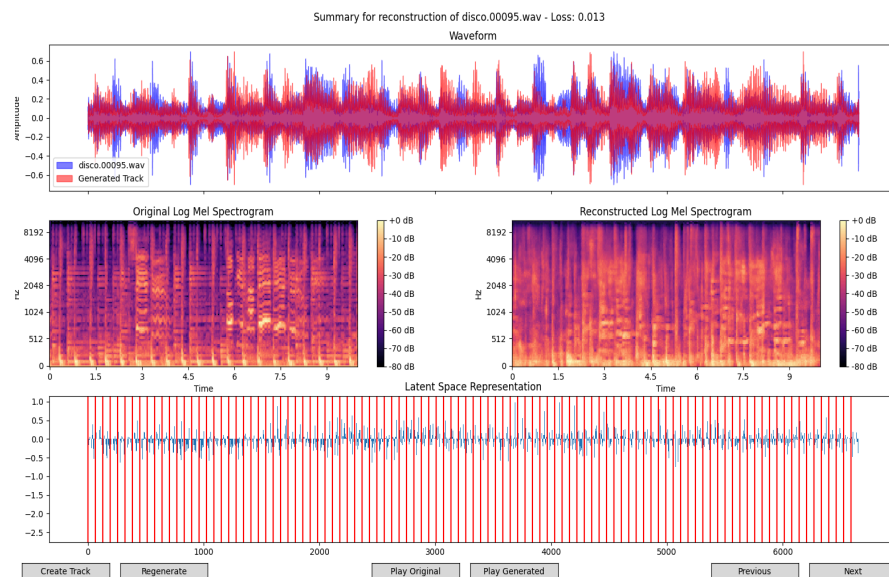
Notre implémentation des modules de VQ-VAE s'inspire grandement de l'adaptation PyTorch^[17] réalisée par **airalcorn2** et disponible publiquement sur GitHub. Elle comprend notamment une classe **SonnetEMA** gérant l'optimisation de l'Exponential Moving Average et un module **VectorQuantizer** qui accepte en entrée les tenseurs produits par l'encodeur et les mappe vers l'espace discret du dictionnaire en choisissant les plus proches selon une distance euclidienne. Cette opération est réalisée grâce à une table discrète de type **nn.Embedding** implémentée par PyTorch. L'encodeur et le décodeur sont des implémentations similaires à celles décrites dans la section précédente et que nous utilisons déjà précédemment, des modules résiduels convolutifs.

La fonction **forward(x, training)** du VectorQuantizer est très intéressante à comprendre, nous l'avons donc retranscrite entièrement avec des commentaires en [Annexe 5](#). Un des points majeurs concerne le **Straight-Through Estimator**. Lors de notre première implémentation d'un VQ-VAE, le modèle semblait ne rien apprendre, et il s'est avéré que nous avons omis cette étape essentielle. En effet, tout comme l'aspect aléatoire d'un VAE, la discrétisation n'est pas un processus différentiable, ce qui empêche l'écoulement du gradient. Pour pallier à ce problème, l'approximation $z_e(x) \approx z_q(x)$ est faite, où $z_e(x)$ est le résultat issue de l'encodeur, et $z_q(x)$ est sa version quantifiée. Bien qu'initialement complètement faussée, cette approximation devient de plus en plus proche de la réalité à mesure de l'évolution du dictionnaire de vecteurs discrets et de l'entraînement du réseau de neurones.

Voici les résultats pour le dataset SINEWAVE étudié dans la partie **2. B) 3) c.** Comme on peut le voir sur le spectrogramme log-mel de droite, la qualité de reconstruction est bien meilleure que celle présentée plus tôt avec un VAE convolutif classique avec presque aucun bruit:



Nous avons également entraîné un modèle similaire sur le dataset GTZAN, une collection de 1000 extraits musicaux que nous avons découpé en passages de 10 secondes. Voici un résultat de reconstruction pour l'un de ces extraits:



Pour terminer, voici un exemple de génération d'audio en tirant aléatoirement un vecteur d'entiers correspondant aux identifiants des vecteurs discrets du dictionnaire.

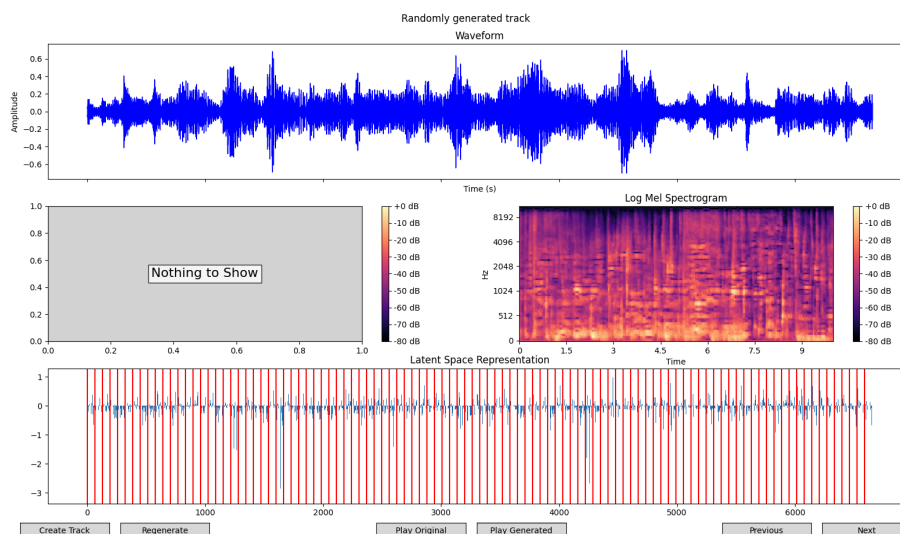


Figure 18, 19, 20: Démonstration des modèles VQ-VAE pour la génération de spectrogrammes

D. Systèmes annexes et interfaces graphiques

Afin de compléter notre solution technique, nous avons développé une interface web interactive permettant aux utilisateurs de générer des mélodies en utilisant nos modèles VAE et d'explorer des démonstrations pré entraînées. Notre interface, combinant React pour le front-end et Flask pour le back-end, s'appuie sur les API que nous avons développées tout au long du projet pour offrir une expérience intuitive et visuelle, tout en facilitant la sélection des modèles.

Notre projet se compose de deux modules principaux:

- Un **serveur Flask** nous permet de créer des routes vers nos modèles entraînés pour des requêtes REST de génération audio.
- Une application **NodeJS et React** se charge de la partie visuelle accessible par l'utilisateur avec une interaction intuitive des modèles, facilitant la soumission de requêtes et le téléchargement de fichiers.

Les échanges entre le front-end et le back-end s'effectuent via des requêtes HTTP asynchrones, permettant une mise à jour dynamique des éléments sans rechargement de page.

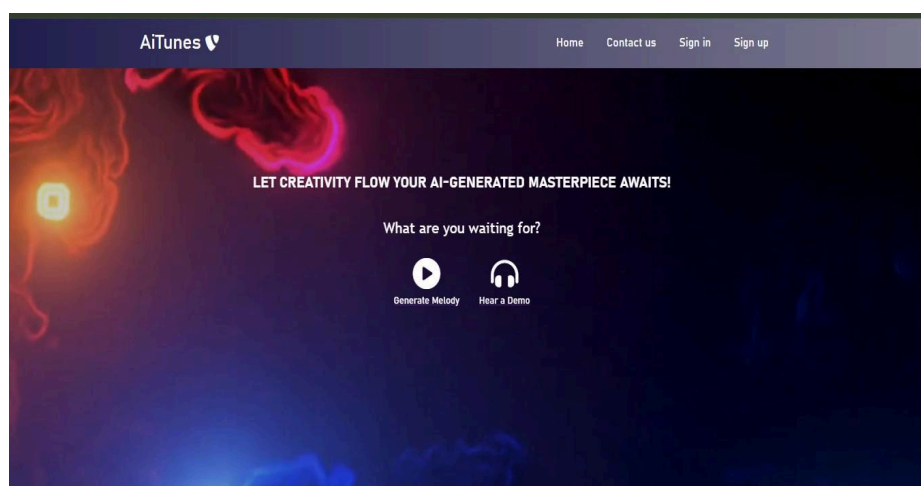


Figure 21: Page d'accueil de l'interface Web du projet AiTunes

La page d'accueil présente une vue synthétique du projet, accompagnée d'un bouton **Get Started** redirigeant vers l'interface de génération de mélodies. Celle-ci propose :

- Un menu déroulant pour choisir parmi les modèles entraînés.
- Dès la sélection d'un modèle, une description personnalisable de ses forces et caractéristiques s'affiche.
- Un clic sur le bouton « *Generate* » déclenche la génération d'un fichier **wav**, qui sera sauvegardé sur le l'appareil de l'utilisateur.

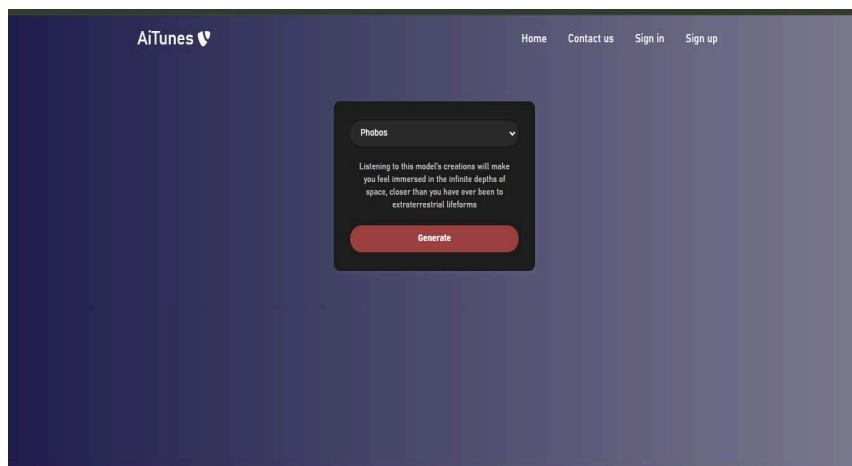


Figure 22: Page de sélection du modèle

Nous avons également développé des pages pour l'inscription et la connexion des utilisateurs, bien que la partie back-end ne soit pas encore fonctionnelle:

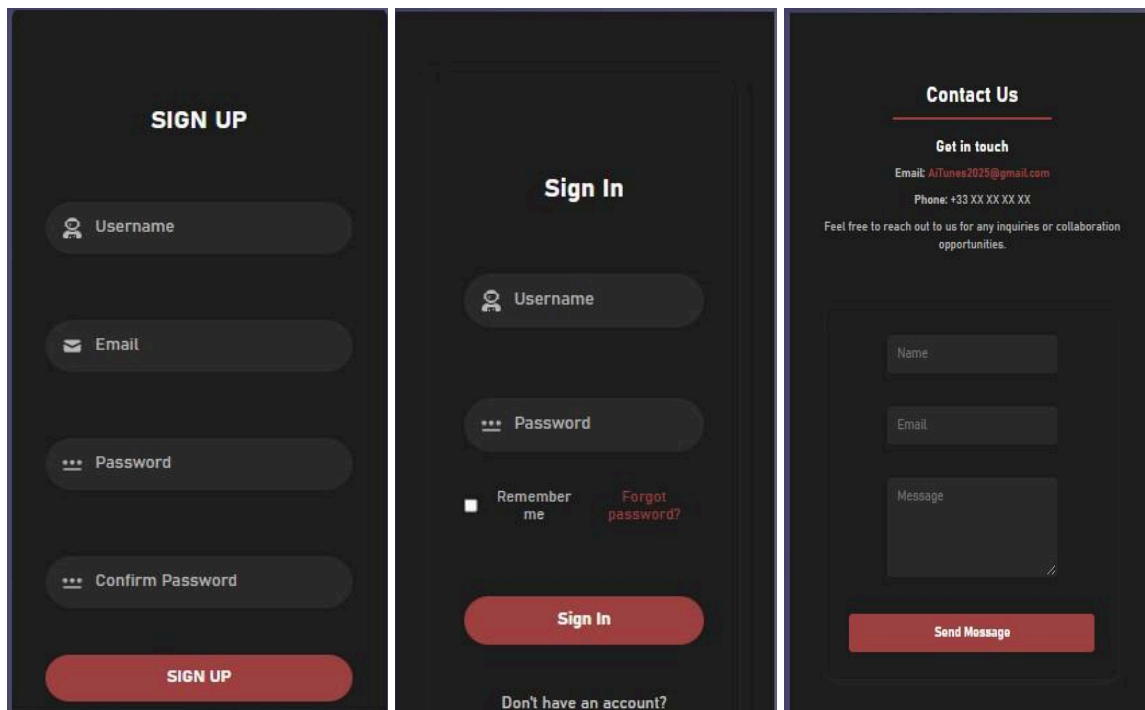


Figure 23: Pages de connexion et de contact

IV. PERSPECTIVES D'ÉVOLUTION

Notre projet AiTunes a atteint ses premiers objectifs en termes de recherche et de découverte des auto-encodeurs appliqués à la génération musicale. Nous avons posé les bases pour la création et l'exploitation rapide de modèles grâce au développement de systèmes flexibles. En expérimentant avec de nombreuses innovations comme l'augmentation linéaire de l'importance de la KL Divergence^[18] ou encore les fonctions d'évaluation perceptuelles^[19], cela nous a ouvert à de nouvelles perspectives d'évolution pour ce projet.

A. Applications génératives

Il serait notamment intéressant d'appliquer les modèles VQ-VAE sur un dataset plus large que GTZAN. Nous avons commencé à faire quelques tests avec le dataset FMA^[20] mais les temps de calculs étant trop élevés, nous n'avons pas réussi à obtenir de résultats significatifs. Une autre idée que nous souhaitons mettre en œuvre est la modification de la voix ou du genre d'une musique. Un exemple serait d'entraîner un modèle sur une base de données de morceaux de violons, puis de passer en entrée un morceau enregistré à la flûte ou au piano. Enfin, nous pourrions considérer une idée similaire pour une application artistique visuelle, où un modèle reçoit le contenu d'une image sur laquelle l'utilisateur dessine. Le résultat produit par le décodeur serait alors conditionné par le type de données sur lequel il a été entraîné et pourrait donc créer des résultats intéressants.

Avec des puissances de calcul encore plus importantes, nous aurions également aimé poursuivre avec l'implémentation de réseaux plus complexes ayant fait leurs preuves, comme le VQ-VAE 2 utilisé par Jukebox qui est constitué d'une suite de modèles VQ-VAE itératifs.

B. Enrichissement de la plateforme web

Nous sommes très satisfaits des efforts que nous avons fait sur la partie des interfaces visuelles. Nous pourrions apporter de nouvelles fonctionnalités à la partie web, avec notamment les idées suivantes:

- **Développement d'un mode collaboratif**, avec la mise en place d'un espace partagé où plusieurs utilisateurs pourraient construire des morceaux, commenter des générations et partager leurs créations.
- **Intégration d'une bibliothèque de samples**, un catalogue plus complet et varié de samples alimenté par les utilisateurs de samples pour donner un meilleur aperçu des possibilités avec AiTunes.
- **Création d'un widget** qui pourrait être intégré à une page web sous forme d'iframe pour jouer de la musique en continu.

Pour ce qui est du back-end:

- **Ajout d'un système d'authentification et d'un espace personnel**: finalisation du système de connexion afin de proposer des profils utilisateurs avec historique, paramètres personnalisés
- **Déploiement avec Docker** afin de faciliter la mise en production et l'accessibilité multiplateforme.
- **Optimisation du temps de réponse** grâce à une gestion avancée du cache pour les requêtes répétées et la génération asynchrone d'audio.

V. CONCLUSION ET MOT DE FIN

Nos projets passés respectifs en intelligence artificielle nous ont grandement accompagné dans cette découverte du domaine des réseaux de neurones génératifs. En combinant la théorie, le développement et les efforts de modélisation et de visualisation, nous avons appris en équipe une grande quantité de concepts qui nous seront très utiles dans nos futurs travaux. Au-delà des aspects techniques, ce travail nous a confronté à des choix importants en termes d'architecture, de traitement de données et d'expérience utilisateur. Nous sommes très satisfaits des systèmes que nous avons développés et du cheminement que nous avons adopté pour y arriver, même si nous aurions aimé obtenir des résultats génératifs plus concluants.

Même si certaines fonctionnalités restent à améliorer ou à compléter, le projet a atteint ses objectifs principaux et ouvre la voie à de nombreuses évolutions. Il constitue une base solide sur laquelle bâtir des développements futurs, plus poussés et encore plus interactifs. Ce projet

marque pour nous un véritable point de départ . Il nous a donné envie de continuer à explorer les liens entre art et technologie et de pousser plus loin les limites de ce que l'intelligence artificielle peut apporter à la création.

BIBLIOGRAPHIE

- [1]: <https://magenta.tensorflow.org/music-vae>
- [2]: <https://datascientest.com/stylegan>
- [3]: <https://openai.com/index/jukebox/>
- [4]: <https://github.com/google-deepmind/sonnet/blob/v2/sonnet/src/nets/vqvae.py>
- [5]: <https://github.com/MisTurtle/AiTunes.git>
- [6]: https://github.com/meryemidboucair/AiTunes_Web_Project.git
- [7]: http://outilsrecherche.over-blog.com/pages/Presentations_3_Echelle_de_Mel-3122565.html
- [8]: <https://optimization.cbe.cornell.edu/index.php?title=Adam>
- [9]: https://www.youtube.com/watch?v=lvNxAeliNKE&ab_channel=MisTurtle
- [10]: <https://irma.math.unistra.fr/~franck/cours/SciML/output/html/chapAPsec5.html>
- [11]: <https://www.kaggle.com/>
- [12]: <https://www.ultralytics.com/fr/glossary/vanishing-gradient>
- [13]: https://www.researchgate.net/figure/Larchitecture-dun-bloc-residuel-13-c-2016-IEEE_fig3_367453093
- [14]: <https://paperswithcode.com/method/vq-vae>
- [15]: <https://github.com/google-deepmind/sonnet/blob/v2/sonnet/src/nets/vqvae.py>
- [16]: <https://web.archive.org/web/20241009222543/https://ovgu-ailab.github.io/blog/methods/2024/05/28/vqvae-compression.html>
- [17]: <https://github.com/airalcorn2/vqvae-pytorch/blob/master/vqvae.py>
- [18]: <https://www.microsoft.com/en-us/research/blog/less-pain-more-gain-a-simple-method-for-vae-training-with-less-of-that-kl-vanishing-agony/>
- [19]: <https://deepai.org/machine-learning-glossary-and-terms/perceptual-loss-function>
- [20]: <https://www.kaggle.com/datasets/noahbadoa/fma-dataset-100k-music-wav-files>

ANNEXE 1 : Fonction d'entraînement et d'évaluation

```
def train(self, epochs: int):
    self._model.train(True) # Passe le modèle en mode entraînement
    self._support.add_total_epochs(epochs) # Met à jour le nombre d'époques appliquées au modèle
    self._support.log(f"A training session of {epochs} epochs is about to start...") # Affichage

    for _ in range(epochs):
        # Extraction des lots mélangés par le générateur self.next_batch
        for batch, *extra in self.next_batch(training=True, lookup_labels=False):
            self._optimizer.zero_grad() # Réinitialisation du gradient

            # Prédiction du modèle sur le lot de données actuel
            embedding, prediction, *args = self._model(batch, training=True)
            # Evaluation de la fonction objectif self._loss_criterion
            combined_loss, *loss_components = self._loss_criterion(prediction, batch, *args)
            # Population de l'arbre des gradients accumulé dans le tenseur issu de l'évaluation précédente
            combined_loss.backward()

            # Sécurité: Arrête automatiquement l'entraînement si un NaN est détecté dans les poids du modèle
            if torch.isnan(combined_loss):
                self.crash_nans()

            # Modification des poids du réseau en fonction des gradients calculés par combined_loss.backward()
            self._optimizer.step()
            # Enregistrement et affichage du résultat intermédiaire
            self._support.add_batch_result(combined_loss.item(), *loss_components).log_training_loss()

        # Affichage de la perte moyenne de l'époque dans la console et préparatifs pour l'époque suivante
        self._support.log_training_loss(ended=True).next_epoch()
        # Sauvegarde le modèle (rien ne se passe si ce n'est pas le moment de réaliser la sauvegarde)
        self._support.perform_save(self._save_weights)

    self._support.trained = True

def evaluate(self):
    self._support.log("An evaluation session of the model is about to start...")
    self._support.blank_epoch() # Réinitialisation pour une nouvelle époque
    self._model.eval() # Passe le modèle en mode évaluation

    # Indique qu'un arbre des gradients n'est pas nécessaire puisqu'aucune rétropropagation ne sera réalisée
    with torch.no_grad():
        # Extraction des lots mélangés par le générateur self.next_batch
        for batch, *extra in self.next_batch(training=False, lookup_labels=False):
            # Prédiction du modèle sur le lot de données actuel
            embedding, prediction, *args = self._model(batch, training=False)
            # Evaluation de la fonction objectif self._loss_criterion
            batch_loss, *loss_components = self._loss_criterion(prediction, batch, *args)
            # Application de fonctions annexes, potentiellement pour une visualisation des sorties
            self.apply_middleware(batch, prediction, embedding, extra, args)
            # Enregistrement et affichage du résultat intermédiaire
            self._support.add_batch_result(batch_loss, *loss_components).log_running_loss("Evaluation")
        # Affichage de la perte moyenne de l'époque d'évaluation dans la console
        self._support.log_running_loss("Evaluation", True, False)
```


ANNEXE 2 : Implémentation de la classe LinearExperiment

```
import math
import torch
import numpy as np

from aitunes.experiments import AutoencoderExperiment

class LinearExperiment(AutoencoderExperiment):

    @staticmethod
    def expand_3d_vector(val: np.array):
        if not isinstance(val, np.ndarray):
            val = np.array(val)
        return np.column_stack((
            val[:, 0]**2,           # Terme quadratique
            np.sin(val[:, 1] * val[:, 2]), # Sinus du produit
            np.log(1 + np.abs(val[:, 0])), # Transformation logarithmique
            val[:, 1] * val[:, 2],   # Multiplication
            np.exp(val[:, 2]) / np.exp(1) # Exponentiel
        ))

    def __init__(self, model, weights_path, loss, optimizer):
        super().__init__("5D VECTORS", model, weights_path, loss, optimizer)
        # Création des datasets d'entraînement et d'évaluation
        self.training_data = self.expand_3d_vector(np.random.rand(2000, 3))
        self.evaluation_data = self.expand_3d_vector(np.random.rand(50, 3))

    @property
    def batch_size(self) -> int:
        return 32 # Nombre de vecteurs par lot, requis plus tard

    @property
    def batch_per_epoch(self) -> int:
        return int(math.ceil(self.training_data.shape[0] / self.batch_size))

    def next_batch(self, training, lookup_labels: bool = False):
        dataset = self.training_data if training else self.evaluation_data
        np.random.shuffle(dataset)
        for i in range(dataset.shape[0] // self.batch_size):
            yield torch.tensor(dataset[i * self.batch_size:(i + 1) * self.batch_size], dtype=torch.float32), None

    def interactive_evaluation(self):
        # [ Retiré pour plus de clarté ]
        pass
```

ANNEXE 3: Auto-Encodeur variationnel avec PyTorch

```
def kl_loss(mu, log_var, reduce=True):
    # Sécurité pour éviter l'explosion du gradient à cause de log_var.exp()
    log_var = torch.clamp(log_var, min=-10, max=10)
    kl_sum = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
    return kl_sum / mu.size(0),

def reparameterize(mu, log_var):
    sigma = torch.exp(0.5 * log_var)
    epsilon = torch.randn_like(sigma)
    return mu + sigma * epsilon

class VariationalAutoEncoder(AiTunesAutoencoderModule):
    def __init__(self, input_shape: int, hidden_layers_dimensions: Sequence[int], latent_dimension: int,
decoder_activation: nn.Module = None):
        super().__init__(input_shape, ), True)
        self.dimensions = [input_shape] + list(hidden_layers_dimensions) + [latent_dimension]
        # Encoder jusqu'à la dimension précédant l'espace latent
        self._encoder = VanillaEncoder(dimensions=self.dimensions[:-1])
        # Décodeur depuis l'espace latent
        self._decoder = VanillaDecoder(dimensions=self.dimensions[1:], activation_type=decoder_activation)
        # Mu et LogVar
        self._mu = nn.Linear(self.dimensions[-2], self.dimensions[-1])
        self._log_var = nn.Linear(self.dimensions[-2], self.dimensions[-1])

    def encode(self, x):
        x = self._encoder(x)
        mu = self._mu(x)
        log_var = self._log_var(x)
        return mu, log_var

    def decode(self, z):
        return self._decoder(z)

    def sample(self, n = 1):
        return torch.randn((n, self.dimensions[-1]))

    def forward(self, x, training=False):
        mu, log_var = self.encode(x)
        z = reparameterize(mu, log_var)
        return z, self.decode(z), mu, log_var
```

ANNEXE 4 : Implémentation du VAE convolutif

```
class ConvolutionalDecoder(nn.Module):

    def __init__(self, target_shapes: Sequence[int], conv_channels: Sequence[int], conv_kernels: Sequence[int],
conv_strides: Sequence[int], latent_dimension: int, activation_type: ActivationType = None):
        super().__init__()
        self._target_shapes = target_shapes
        self._channels = conv_channels
        self._kernels = conv_kernels
        self._strides = conv_strides
        self._latent_dimension = latent_dimension
        self._activation_type = activation_type

        self._layers = nn.Sequential()
        self._build_layers()

    def _build_layers(self):
        self._layers.append(nn.Linear(self._latent_dimension, self._channels[0] * np.prod(self._target_shapes[0])))
        self._layers.append(nn.Unflatten(1, (self._channels[0], *self._target_shapes[0])))
        for i in range(len(self._channels) - 1):
            p_h, p_w = compute_transpose_padding(
                kernel_size=self._kernels[i],
                padding=self._kernels[i] // 2,
                stride=self._strides[i],
                input_shape=self._target_shapes[i],
                output_shape=self._target_shapes[i + 1]
            )
            self._layers.append(nn.ConvTranspose2d(
                in_channels=self._channels[i],
                out_channels=self._channels[i + 1],
                kernel_size=self._kernels[i],
                stride=self._strides[i],
                padding=self._kernels[i] // 2,
                output_padding=(p_h, p_w)
            ))
            if i != len(self._channels) - 2:
                self._layers.append(nn.BatchNorm2d(self._channels[i + 1]))
                self._layers.append(nn.ReLU())

        if self._activation_type is not None:
            self._layers.append(self._activation_type())

    def forward(self, x):
        return self._layers(x)

class CVAE(AiTunesAutoencoderModule):

    def __init__(self, input_shape, conv_channels: Sequence[int], conv_kernels: Sequence[int], conv_strides:
Sequence[int], latent_dimension: int, decoder_activation: ActivationType = None):
        super().__init__(input_shape, False)
        self._conv_channels = [input_shape[0]] + list(conv_channels)
        self._conv_kernels = conv_kernels
        self._conv_strides = conv_strides
        self._latent_dimension = latent_dimension

        self._encoder = ConvolutionalEncoder(
            input_shape,
            self._conv_channels,
            self._conv_kernels,
            self._conv_strides,
            self._latent_dimension
        )
        self._decoder = ConvolutionalDecoder(
            self._encoder.shapes[1:-1],
            self._conv_channels[1:-1],
            self._conv_kernels[1:-1],
            self._conv_strides[1:-1],
            self._latent_dimension,
            decoder_activation
        )

    def encode(self, x):
        mu, log_var = self._encoder(x)
        return mu, log_var

    def decode(self, z):
        return self._decoder(z)

    def sample(self, n = 1):
        mu = self._encoder._mu(torch.randn((n, self._encoder.shapes[-1])))
        log_var = self._encoder._log_var((n, torch.randn(self._encoder.shapes[-1])))
        return reparameterize(mu, log_var)

    def forward(self, x, training=False):
        mu, log_var = self.encode(x)
        z = reparameterize(mu, log_var)
        return z, self.decode(z), mu, log_var
```

ANNEXE 5 : Application à un dataset de musiques réelles

```
def forward(self, x, training=False):
    # Dans les commentaires qui suivent -->
    # B: taille du lot ; H et W: hauteur et largeur dans l'espace latent
    # N: dimension des vecteurs caractéristiques ; n: nombre de vecteurs discrets dans le dictionnaire

    # Redimensionner l'entrée pour isoler la taille du lot et celle du code latent,
    # récupérant une liste de vecteurs caractéristiques de la même taille que la table discrète
    flat_x = x.permute(0, 2, 3, 1) # B, H, W, N
    flat_x = flat_x.reshape((flat_x.shape[0], -1, flat_x.shape[-1])) # [B, H x W, N]

    # Calcul de la distance entre chaque vecteur discret et chaque vecteur caractéristique
    lookup_weights = self.embeddings.weight[None, :].repeat((flat_x.shape[0], 1, 1))
    distances = torch.cdist(flat_x, lookup_weights) # [B, H x W, n]

    # Extrait l'id des vecteurs discrets qui sont les plus proches de chacun des vecteur caractéristique
    encoding_indices = distances.argmin(dim=-1) # B, H x W
    # Etape de quantification
    quantized = torch.index_select(self.embeddings.weight, 0, encoding_indices.view(-1)) # B x H x W, N
    flat_x = flat_x.reshape((-1, flat_x.shape[-1])) # B x H x W, Embedding dim

    # Calcul des pertes pour mettre à jour le réseau
    # Codebook loss: Les vecteurs discrets se dirigent vers le résultat de l'encodeur
    self.last_codebook_loss = None if self.ema else torch.mean((quantized - flat_x.detach()) ** 2)
    # Commitment loss: L'encodeur devrait produire des résultats plus proches des vecteurs discrets
    self.last_commitment_loss = torch.mean((flat_x - quantized.detach()) ** 2)

    # Préparatifs pour historiser le degré d'utilisation des vecteurs discrets
    encodings = F.one_hot(encoding_indices.view(-1), num_classes=self.num_embeddings).float() # B x H x W, n
    cluster_count = encodings.sum(dim=0)
    self.overall_embedding_usage += cluster_count # historisation

    if self.ema and training:
        # Entraînement manuel des Sonnet EMA
        with torch.no_grad():
            # Calcul de l'utilisation de chaque vecteur discret
            updated_ema_cluster_size = self.EMA_cluster_counts(cluster_count)
            dw = torch.matmul(flat_x.T, encodings)
            updated_ema_dw = self.EMA_embeddings(dw)

            # Mise à jour progressive de la table de vecteurs discrets
            n = updated_ema_cluster_size.sum()
            updated_ema_cluster_size = ((updated_ema_cluster_size + self.epsilon) / (n + self.num_embeddings *
self.epsilon)) * n)
            normalised_updated_ema_w = updated_ema_dw / updated_ema_cluster_size.unsqueeze(0)
            self.embeddings.weight.copy_(normalised_updated_ema_w.t())

            # Technique du Random Codebook Restart
            if self.random_restart > 0:
                self.used_embeddings += cluster_count
                self.restart_in -= 1
                if self.restart_in == 0:
                    self.restart_in = self.random_restart
                    dead_indices = torch.where(self.used_embeddings < self.restart_threshold)[0]
                    if dead_indices.numel() > 0:
                        # Réinitialisation des entrées qui ne sont pas suffisamment utilisées avec des valeurs du
batch actuel
                        rand_indices = torch.randint(0, flat_x.size(0), (dead_indices.shape[0], ))
                        new_entries = flat_x[rand_indices]
                        self.embeddings.weight[dead_indices] = new_entries
                        # Réinitialisation des données d'utilisation
                        self.used_embeddings = torch.zero(self.used_embeddings)

            # Straight Through Estimation: quantized ~ x
            quantized = flat_x + (quantized - flat_x).detach()
            quantized = quantized.reshape(x.shape) # Batch Size, Channels, Height, Width

            # Perplexity computation: évaluation de la qualité de l'utilisation du codebook
            avg_probs = torch.mean(encodings, 0)
            self.last_perplexity = torch.exp(-torch.sum(avg_probs * torch.log(avg_probs + 1e-10)))

    return quantized
```